

Group 12

Privacy Preserving Collaborative Resource and Task Orchestrator

Software Architecture Document

Version 1.0

Mentor: Mr. Dulaj Dilshan

Group Members:

- 230324D P.M.M.C.S Karunarathna
- 230320L N.C.T Karunanayaka
- 230328T M.A.D Kavinda

Contribution

Member	Task
P.M.M.C.S Karunarathna - 230324D	1 - Introduction 4 - Use-Case View 9 - Data View 12 - References
M.A.D Kavinda - 230328T	3 - Architectural Goals and Constraints 6 - Process View 7 - Deployment View 11 - Quality
N.C.T Karunanayaka - 230320L	2 - Architectural Representation 5 - Logical View 8 - Implementation View 10 - Size and Performance

Table of Contents

1. Introduction
2. Architectural Representation
3. Architectural Goals and Constraints
4. Use-Case View
5. Logical View
6. Process View
 - 6.1 Activity Diagrams
 - 6.1.1 User Authentication and Authorization Process
 - 6.1.2 Project Workspace Setup
 - 6.1.3 Kanban Task Lifecycle Flow
 - 6.1.4 Resource Booking and Conflict Resolution
 - 6.1.5 Resource Maintenance Scheduling
 - 6.1.6 Real-Time Document Synchronization
 - 6.1.7 AI Context Processing Workflow
 - 6.1.8 AI Task Approval Process
 - 6.1.9 Mobile Note Capture and Sync
 - 6.2 Sequence Diagrams
 - 6.2.1 User Authentication & JWT Generation
 - 6.2.2 Project Creation & Role Assignment
 - 6.2.3 Kanban Task Management
 - 6.2.4 Resource Booking with Concurrency Lock
 - 6.2.5 Equipment Catalog Management

6.2.6 Secure Real-Time Chat (WebSockets)

6.2.7 Synchronous Collaborative Document Editing

6.2.8 Background AI Chat Summarization

6.2.9 AI Suggested Task Approval

6.2.10 Mobile Quick-Capture Note Sync

6.2.11 Mobile Push Notification Dispatch

7. Deployment View

8. Implementation View

9. Data View

10. Size and Performance

11. Quality

12. References

1. Introduction

1.1 Purpose

This Software Architecture Document provides a comprehensive architectural overview of the Privacy Preserving Collaborative Resource and Task Orchestrator for Research Environments. It translates the project's requirements into a structured design, ensuring stakeholders and developers share a common understanding of the system's privacy-first architecture, automated context engine, and resource management components.

1.2 Scope

The system is a full-stack, self-hostable Private SaaS platform tailored for small research labs. It provides:

- Interactive Kanban boards for task management.
- A unified dashboard for shared resource scheduling (e.g., booking lab GPUs, workstations).
- A collaborative rich-text editor for meeting minutes.
- An Automated Context Engine (using LangChain) to extract action items and summarize chats.
- Mobile companion app for quick notes and real-time push notifications.
- End-to-end encrypted messaging and data storage.

1.3 Definitions, Acronyms, and Abbreviations

Private SaaS - Single-tenant software deployed on isolated infrastructure.

PI - Principal Investigator.

JWT - JSON Web Token.

LLM - Large Language Model.

Context Engine - AI service summarizing interactions.

1.4 Overview

The remainder of this document details the architectural representations, goals, constraints, use-cases, and subsequent structural views (Logical, Process, Deployment, Implementation, Data).

2. Architectural Representation

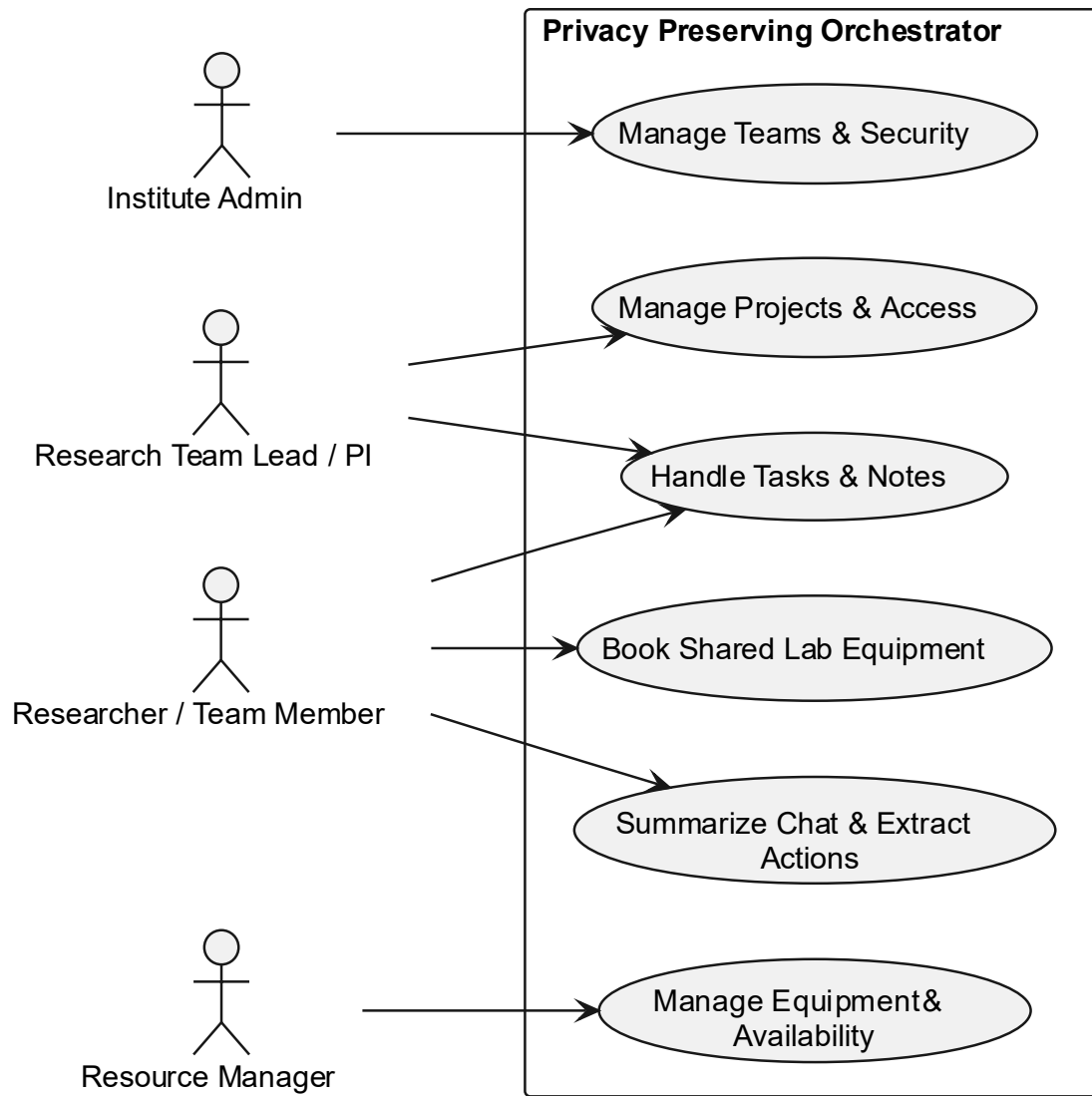
Architectural representation in software engineering is the way we visualize, document, and communicate the structure and behavior of a complex software system.

The most common framework for this is the 4+1 Architectural View Model, which is what was used in the diagrams below:

- Use-Case View (The "+1"): Shows *what* the system does from the user's perspective. It ties all the other views together.
- Logical View: Shows the functional components and how they fit together (useful for system designers).
- Process View: Shows the dynamic flow, concurrency, and real-time behavior (useful for integrators).
- Deployment (Physical) View: Shows how the software maps to physical hardware and servers (useful for system admins).
- Implementation (Development) View: Shows how the actual code is organized and packaged (useful for programmers).

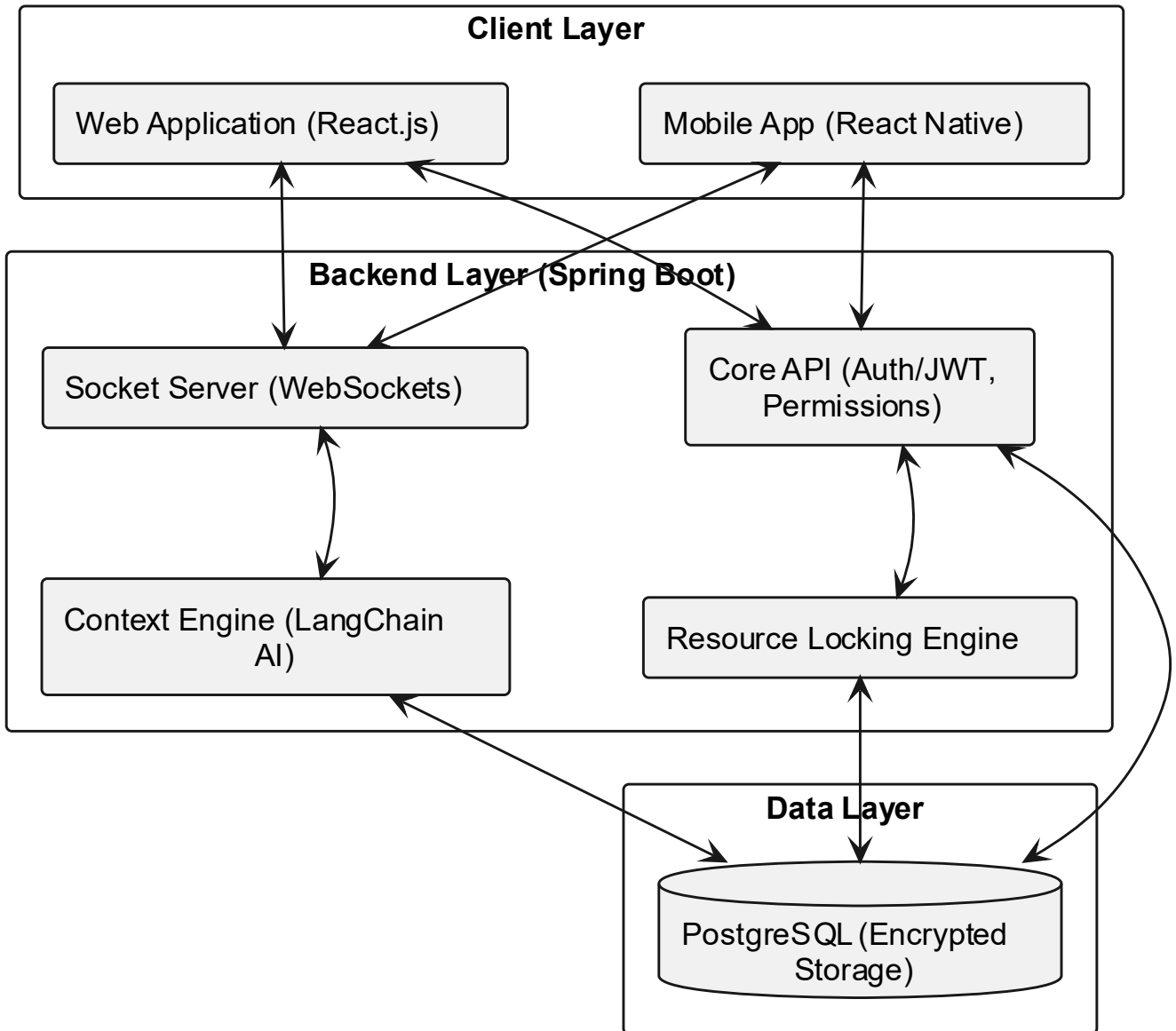
1. Use-Case View

This view describes the system's functionality from the perspective of external actors (users). It validates that the architecture can provide the required functionality.



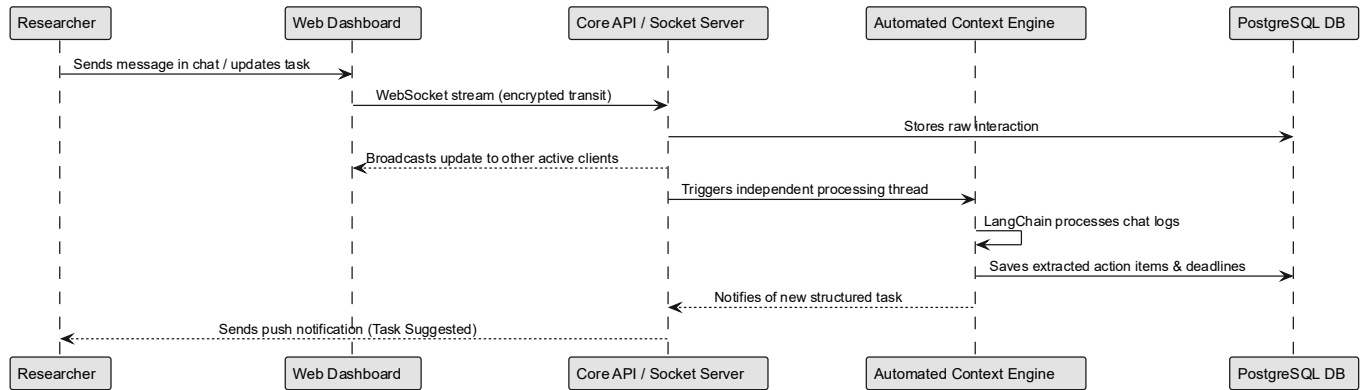
2. Logical View

This view illustrates the functional structure of the system, showing how the system is decomposed into distinct components and services.



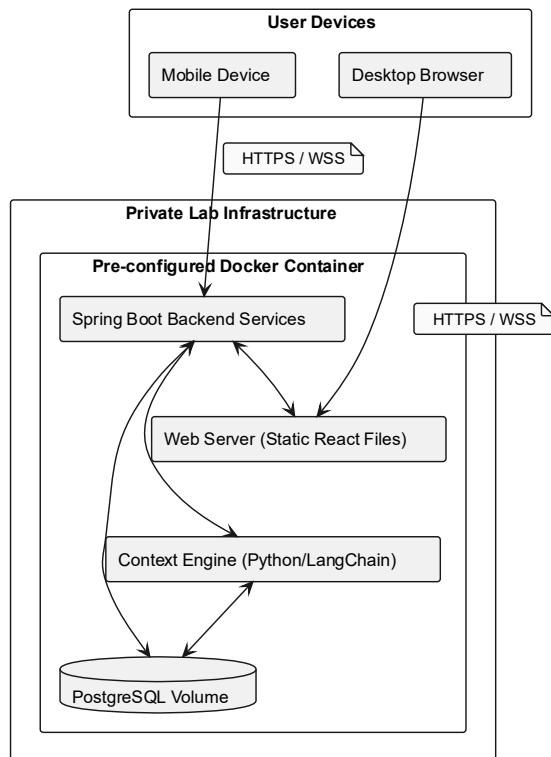
3. Process View

This view captures the dynamic aspects of the system, focusing on runtime behavior, concurrency, and how components communicate.



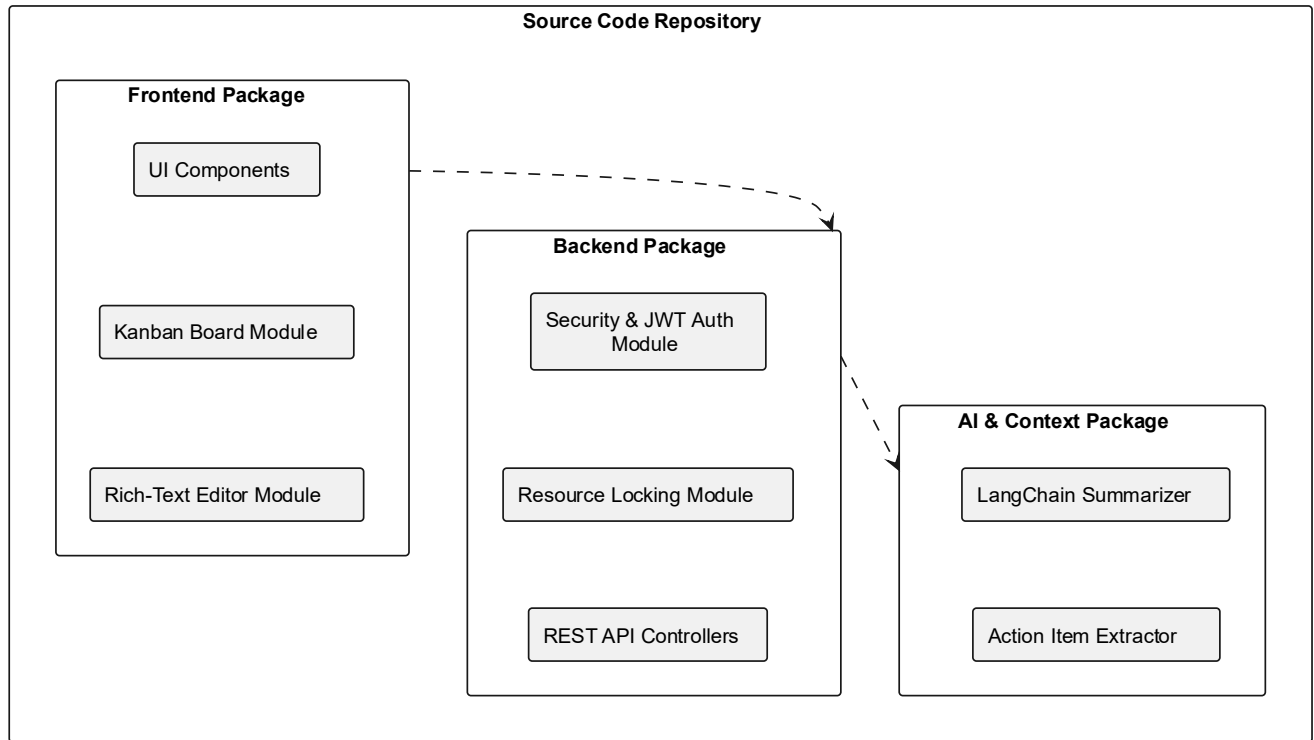
4. Deployment View

This view maps the software components to the physical hardware and execution environments, highlighting the private hosting strategy.



5. Implementation View

This view represents the organization of the actual software modules, packages, and files within the development environment.



3. Architectural Goals and Constraints

This section describes the software requirements, objectives, and constraints that have a significant impact on the system's architecture.

3.1 Architectural Goals

- **Absolute Data Sovereignty (Privacy-First):** The primary goal of the architecture is to protect highly sensitive, pre-publication research intellectual property. The system must operate under a "Privacy-First" mandate, ensuring that data is never exposed to multi-tenant public cloud vulnerabilities.
- **Real-Time Responsiveness:** The architecture must support low-latency, synchronous communication to facilitate live collaborative editing of meeting minutes and instant chat messaging.

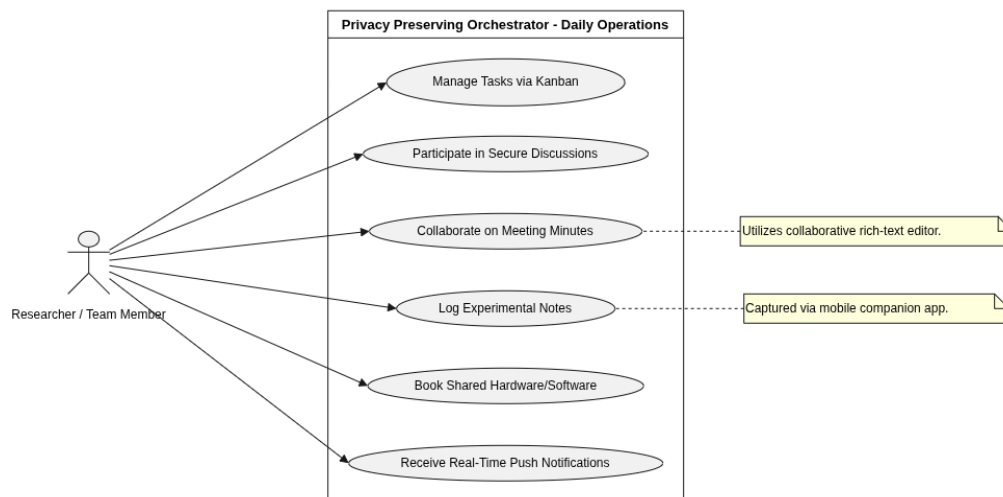
- **Automated Context Processing:** The system must autonomously process natural language data (chat logs) to reduce administrative overhead for researchers, extracting action items and generate summaries in the background.

3.2 Constraints

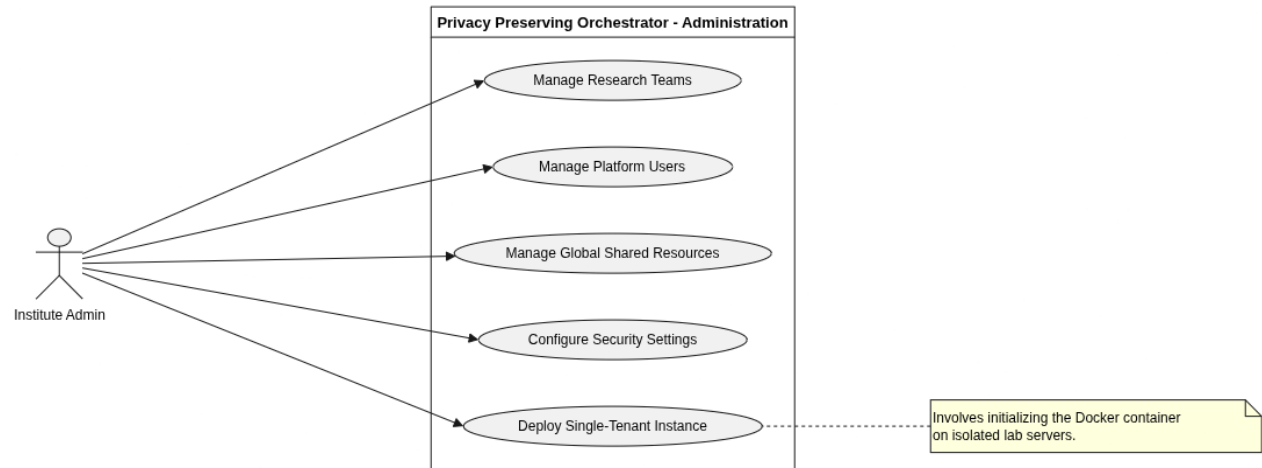
- **Deployment & Containerization Constraint:** To guarantee data sovereignty and allow institutions to self-host the platform, the entire architecture (databases, APIs, and frontends) must be strictly containerized using Docker.
- **Security Constraint:** All sensitive data payloads must be encrypted using the AES-256 encryption standard at the application layer, both at rest and in transit.
- **Localized AI Processing Constraint:** The Automated Context Engine must utilize LangChain to run localized language model processing. Proprietary research chat logs must never be transmitted to external, public AI training endpoints (e.g., standard OpenAI web interfaces).
- **Polyglot Microservices Constraint:** The backend must be decoupled into specialized services (Java Spring Boot for core data, Node.js for WebSockets, Python for AI) rather than a monolith, to handle the distinct technological requirements of each module.
- **Hardware Limitations:** The system must be capable of being deployed on standard private laboratory servers. A minimum of 8GB RAM is recommended to run the local Docker containers concurrently.

4. Use-Case View

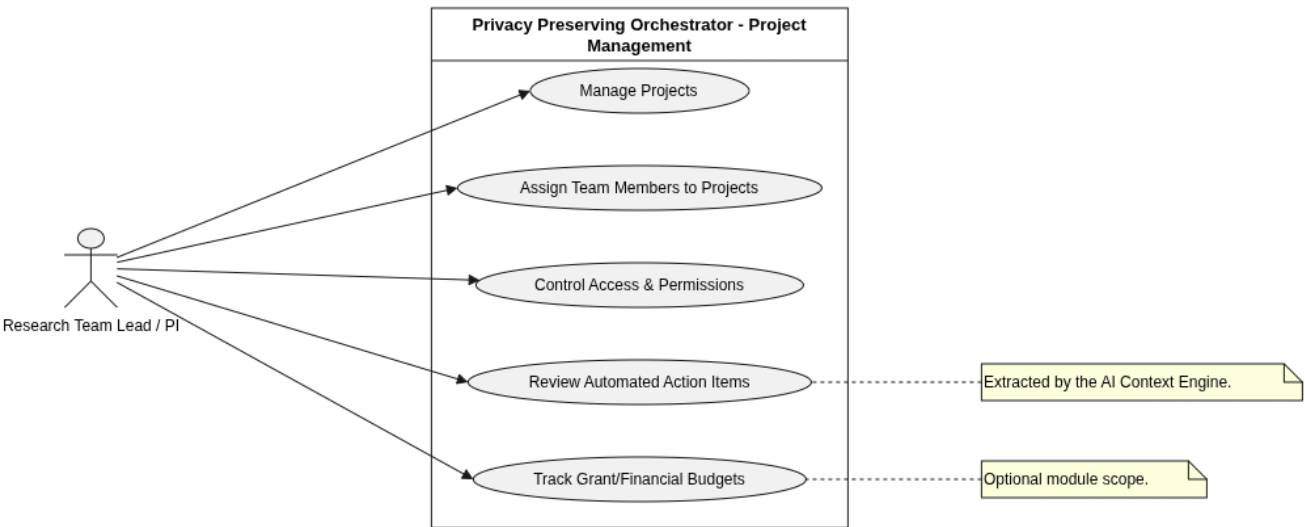
1. Researcher



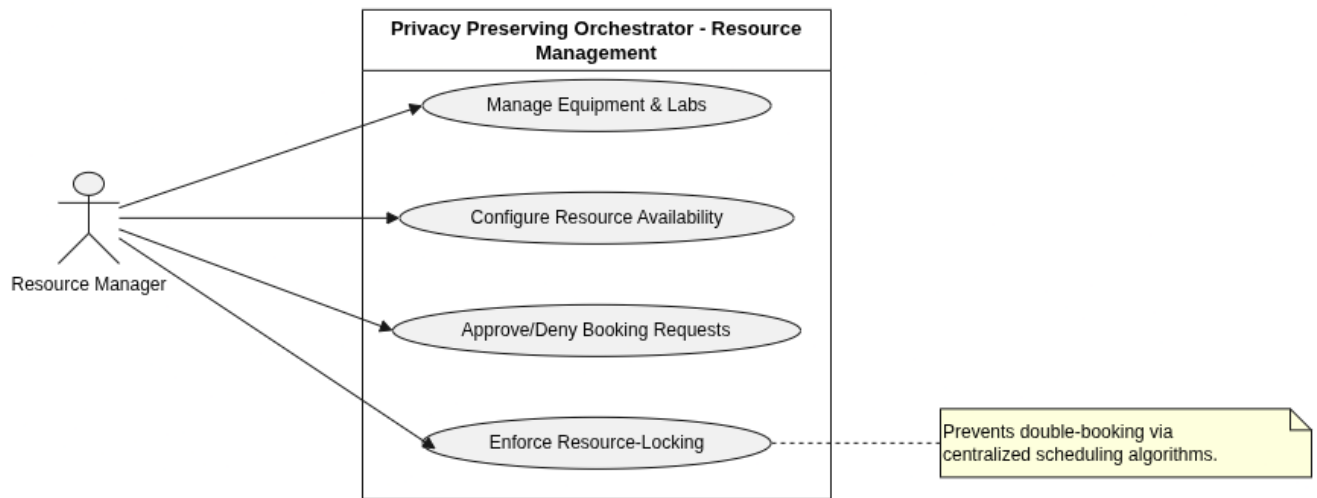
2. Administrator



3. Research Team Lead



4. Resource Manager



4.1 Use-Case Realizations

1. Deploy Single-Tenant Instance

Use case name	Deploy Single-Tenant Instance
Actor	Institute Admin
Description	Initializes the isolated, containerized application and database on local lab servers to ensure data sovereignty.
Preconditions	Admin has access to the private lab infrastructure and Docker.
Main flow	1. Admin configures deployment parameters for the isolated environment. 2. Admin executes the pre-configured Docker container setup. 3. System provisions the isolated PostgreSQL database and backend.

Successful end/post condition	Private SaaS instance runs locally without external third-party access.
Fail end/post condition	Deployment fails, and the system remains offline.
Extensions	N/A

2. Manage Platform Users

Use case name	Manage Platform Users
Actor	Institute Admin
Description	Adds, edits, or deactivates user accounts across the entire isolated platform.
Preconditions	Admin is logged into the global administration dashboard.
Main flow	1. Admin navigates to the User Management section. 2. Admin inputs new user details or selects an existing user to modify. 3. Admin assigns a global role (e.g., Resource Manager, Researcher). 4. System updates the PostgreSQL database.
Successful end/post condition	User account is successfully created or updated.
Fail end/post condition	Database update fails, returning an error message.
Extensions	N/A

3. Manage Research Teams

Use case name	Manage Research Teams
Actor	Institute Admin
Description	Creates top-level research team groups and assigns the Team Leads / Research Team Leads to them.
Preconditions	The users who will act as Team Leads must already exist in the system.
Main flow	1. Admin selects "Create Team". 2. Admin designates a name and assigns a Research Team Lead / Research Team Lead. 3. Core API persists in the new team hierarchy.
Successful end/post condition	Team is established, allowing the Research Team Lead to start creating projects.
Fail end/post condition	Team creation fails due to missing user data.
Extensions	N/A

4. Register / Login

Use case name	Register / Login
Actor	Institute Admin, Research Team Lead / PI, Researcher, Resource Manager
Description	Allows all system actors to securely authenticate into the isolated environment.
Preconditions	The single-tenant instance is deployed and running on the lab server.

Main flow	1. User navigates to the login dashboard. 2. User inputs credentials. 3. The Core API validates the credentials and checks role permissions. 4. System returns a secure JWT. 5. User is redirected to their role-specific workspace.
Successful end/post condition	User securely accesses the encrypted environment.
Fail end/post condition	Authentication fails; access is denied.
Extensions	N/A

5. Create New Project

Use case name	Create New Project
Actor	Research Team Lead / PI
Description	Initializes a new research project workspace within the platform.
Preconditions	User must have Team Lead / PI permissions.
Main flow	1. Team Lead selects "New Project" from the dashboard. 2. Enters project details and privacy parameters. 3. Core API persists the data in PostgreSQL. 4. System generates an isolated Kanban board and document hub for the project.
Successful end/post condition	A new encrypted project workspace is created.
Fail end/post condition	Project creation fails due to database error.
Extensions	N/A

6. Assign Team Members to Projects

Use case name	Assign Team Members to Projects
Actor	Research Team Lead / PI
Description	Grants specific researchers access to a project's tasks, chats, and documents.
Preconditions	Project exists and the users are registered on the platform.
Main flow	1. Team Lead opens project settings. 2. Selects researchers to add to the workspace. 3. Assigns specific access controls. 4. System updates project permissions in the Core API.
Successful end/post condition	Researchers can view and interact with the project's data.
Fail end/post condition	System throws a permission error.
Extensions	N/A

7. Manage Project Permissions

Use case name	Manage Project Permissions
Actor	Research Team Lead / PI
Description	Controls granular access rights within a specific project, determining who can edit documents or delete tasks.
Preconditions	Project exists and team members are assigned.
Main flow	1. Team Lead accesses the project's access control settings.

	2. Adjusts read/write permissions for individual team members. 3. System updates the access control list in the database.
Successful end/post condition	Permissions are strictly enforced by the Core API.
Fail end/post condition	Permission update fails to save.
Extensions	N/A

8. Create Task

Use case name	Create Task
Actor	Researcher / Team Member, Research Team Lead / PI
Description	Manually adds a new actionable item to the project's Kanban board.
Preconditions	User is authenticated in the project workspace.
Main flow	1. User clicks to add a new card on the Kanban board. 2. Inputs title, description, and assigns a team member. 3. Core API persists the task.
Successful end/post condition	Task appears on the board for all project members.
Fail end/post condition	Task creation fails.
Extensions	N/A

9. Update Task Status

Use case name	Update Task Status
Actor	Researcher / Team Member
Description	Moves a task across the Kanban board to reflect progress (e.g., To Do -> In Progress).
Preconditions	Tasks exist on the board.
Main flow	1. User drags and drops the task card on the interactive board. 2. Web frontend sends status update to the Core API. 3. System updates the database.
Successful end/post condition	Task status is updated and synchronized via Socket Server.
Fail end/post condition	UI reverts task position due to sync failure.
Extensions	Can trigger push notifications to the Team Lead.

10. Manage Tasks via Kanban

Use case name	Manage Tasks via Kanban
Actor	Researcher / Team Member, Research Team Lead / PI
Description	Users interact with unified Kanban boards for tracking project tasks and deadlines.
Preconditions	User is authenticated and assigned to an active project.
Main flow	1. User opens the web dashboard.

	2. User selects the interactive Kanban board. 3. User creates, updates, or moves a task card. 4. Socket Server broadcasts real-time updates to all connected clients.
Successful end/post condition	Task is updated and instantly synced across the team.
Fail end/post condition	Task update fails to save due to connection loss.
Extensions	Task deadlines can be auto-suggested by the Context Service.

11. Manage Global Shared Resources

Use case name	Manage Global Shared Resources
Actor	Institute Admin, Resource Manager
Description	Adds, edits, or removes shared hardware, lab equipment, and software assets from the central system.
Preconditions	User has Admin or Resource Manager privileges.
Main flow	1. User navigates to the resource management dashboard. 2. Inputs details for a new asset (e.g., high-performance workstation, specialized sensor). 3. Sets availability parameters. 4. System updates the resource registry.
Successful end/post condition	The new equipment is available for researchers to book.
Fail end/post condition	Asset registration fails.
Extensions	N/A

12. Configure Resource Availability

Use case name	Configure Resource Availability
Actor	Resource Manager
Description	Sets operational hours or blocks out maintenance times for shared lab equipment.
Preconditions	Equipment is registered in the system.
Main flow	1. Manager selects a specific resource (e.g., meeting room, sensor). 2. Manager blocks out specific dates/times. 3. System updates the scheduling dashboard.
Successful end/post condition	Equipment cannot be booked during the specified downtime.
Fail end/post condition	Schedule update fails to save.
Extensions	N/A

13. Book Shared Hardware/Software

Use case name	Book Shared Hardware/Software
Actor	Researcher / Team Member
Description	Schedules usage of shared lab equipment or workstations via the dashboard to prevent conflicts.
Preconditions	User has appropriate access rights to request the resource.

Main flow	1. User accesses the resource scheduling dashboard. 2. User selects desired hardware/software and time slot. 3. System checks for overlapping bookings using the resource-locking algorithm. 4. System reserves the slot pending manager approval.
Successful end/post condition	Resource is provisionally booked without double-booking.
Fail end/post condition	System rejects request immediately due to an existing time conflict.
Extensions	N/A

14. Approve/Deny Booking Requests

Use case name	Approve/Deny Booking Requests
Actor	Resource Manager
Description	Reviews and manages researcher requests for shared hardware or software assets.
Preconditions	A pending equipment booking request exists.
Main flow	1. Manager receives notification of a booking request. 2. Manager reviews the schedule and user clearance. 3. Manager approves or denies the request. 4. System applies the resource lock and notifies the researcher.
Successful end/post condition	Booking is finalized and locked in the centralized schedule.
Fail end/post condition	Request processing fails or times out.

Extensions	N/A
-------------------	-----

15. Participate in Secure Discussions

Use case name	Participate in Secure Discussions
Actor	Researcher / Team Member, Research Team Lead / PI
Description	Team members engage in end-to-end encrypted chat threads directly linked to project tasks or codebases.
Preconditions	Users must be part of the same project team.
Main flow	1. User opens a project discussion thread. 2. Types and sends a message. 3. Socket Server encrypts and broadcasts the message in real-time to other active clients.
Successful end/post condition	Message is delivered and stored securely.
Fail end/post condition	Message fails to send; user is notified of connection error.
Extensions	Messages can be parsed by the Context Service for action item extraction.

16. Collaborate on Meeting Minutes

Use case name	Collaborate on Meeting Minutes
Actor	Researcher / Team Member, Research Team Lead / PI
Description	Multiple users edit a shared document simultaneously within the centralized documentation hub.

Preconditions	Users have access to the rich-text editor for the specific document.
Main flow	1. Users open the same meeting minutes document via the web application. 2. Users input text simultaneously. 3. Socket Server manages operational transforms to sync edits instantly across all clients.
Successful end/post condition	Document reflects all real-time changes without version conflicts.
Fail end/post condition	Sync error occurs; users are prompted to reload.
Extensions	N/A

17. Log Experimental Notes

Use case name	Log Experimental Notes
Actor	Researcher / Team Member
Description	Uses the mobile companion app to quickly capture and log research notes on the go.
Preconditions	User is logged into the secure mobile application.
Main flow	1. User opens the quick-capture interface on the mobile app. 2. User inputs text or data regarding an experiment. 3. System encrypts the data and transmits it to the Core API. 4. System saves the note to the centralized documentation hub.
Successful end/post condition	Note is securely saved and accessible via the web platform.

Fail end/post condition	Network error prevents syncing; data is cached securely until reconnection.
Extensions	Notes can trigger real-time push notifications for tagged team members.

18. Upload Sensitive Research Data

Use case name	Upload Sensitive Research Data
Actor	Researcher / Team Member, Research Team Lead / PI
Description	Securely uploads proprietary files or documentation to the encrypted platform.
Preconditions	User has write access to the documentation hub.
Main flow	1. User selects a file to upload. 2. System encrypts the file during transit. 3. File is stored securely at rest within the isolated backend.
Successful end/post condition	File is safely stored without third-party exposure.
Fail end/post condition	Upload times out or file format is rejected.
Extensions	Can be routed to integrated external 3rd-party cloud storage (e.g., Google Drive/Dropbox) if configured.

19. Generate Automated Meeting Summaries

Use case name	Generate Automated Meeting Summaries
Actor	System (Context Service)

Description	The AI backend processes collaborative meeting minutes and chat logs to generate structural summaries.
Preconditions	A meeting document or discussion thread has sufficient text data.
Main flow	1. Context Service pulls text from the Core API. 2. LangChain processes the text to identify key topics and decisions. 3. System formats a summary. 4. Summary is attached to the project documentation.
Successful end/post condition	A concise, automated summary is available for researchers.
Fail end/post condition	Service fails to parse the text contextually.
Extensions	N/A

20. Review Automated Action Items

Use case name	Review Automated Action Items
Actor	Research Team Lead / PI
Description	Reviews task deadlines and meeting summaries generated automatically by the AI Context Engine.
Preconditions	Context Service has processed recent chat logs or meeting minutes.
Main flow	1. AI Context Engine summarizes long discussion threads using LangChain. 2. System extracts structured action items and deadline suggestions. 3. Team Lead opens the project dashboard.

	4. Team Lead approves the AI-suggested tasks to add them to the Kanban board.
Successful end/post condition	Automated action items are converted into tracked tasks.
Fail end/post condition	AI fails to extract meaningful data and returns an empty summary.
Extensions	Team Lead manually edits the suggested summary before final approval.

21. Receive Real-Time Push Notifications

Use case name	Receive Real-Time Push Notifications
Actor	Researcher / Team Member, Research Team Lead / PI
Description	Delivers instant alerts to the mobile companion app for task updates, mentions, or booking approvals.
Preconditions	User has the mobile app installed and notifications enabled.
Main flow	1. A system event occurs (e.g., a manager approves a lab GPU booking). 2. Core API triggers a notification payload. 3. Payload is sent to the mobile device. 4. Mobile app displays the alert to the user.
Successful end/post condition	User is successfully alerted to the update.
Fail end/post condition	Notification drops due to mobile offline status.
Extensions	N/A

23. Integrate External Cloud Storage

Use case name	Integrate External Cloud Storage
Actor	Institute Admin, Research Team Lead / PI
Description	Links the system to external 3rd-party storage (e.g., Google Drive or Dropbox) for teams that opt out of local-only storage.
Preconditions	The optional cloud storage module is enabled.
Main flow	1. User navigates to storage integrations. 2. User authenticates with the 3rd-party provider (Google/Dropbox) via OAuth. 3. System securely stores the integration tokens.
Successful end/post condition	Platform can push/pull documents to the external cloud while maintaining privacy.
Fail end/post condition	OAuth handshake fails or is rejected by the provider.
Extensions	N/A

24. Generate Automated Meeting Summaries

Use case name	Generate Automated Meeting Summaries
Actor	System (Context Service)
Description	The AI backend processes collaborative meeting minutes and chat logs to generate structural summaries.
Preconditions	A meeting document or discussion thread has sufficient text data.

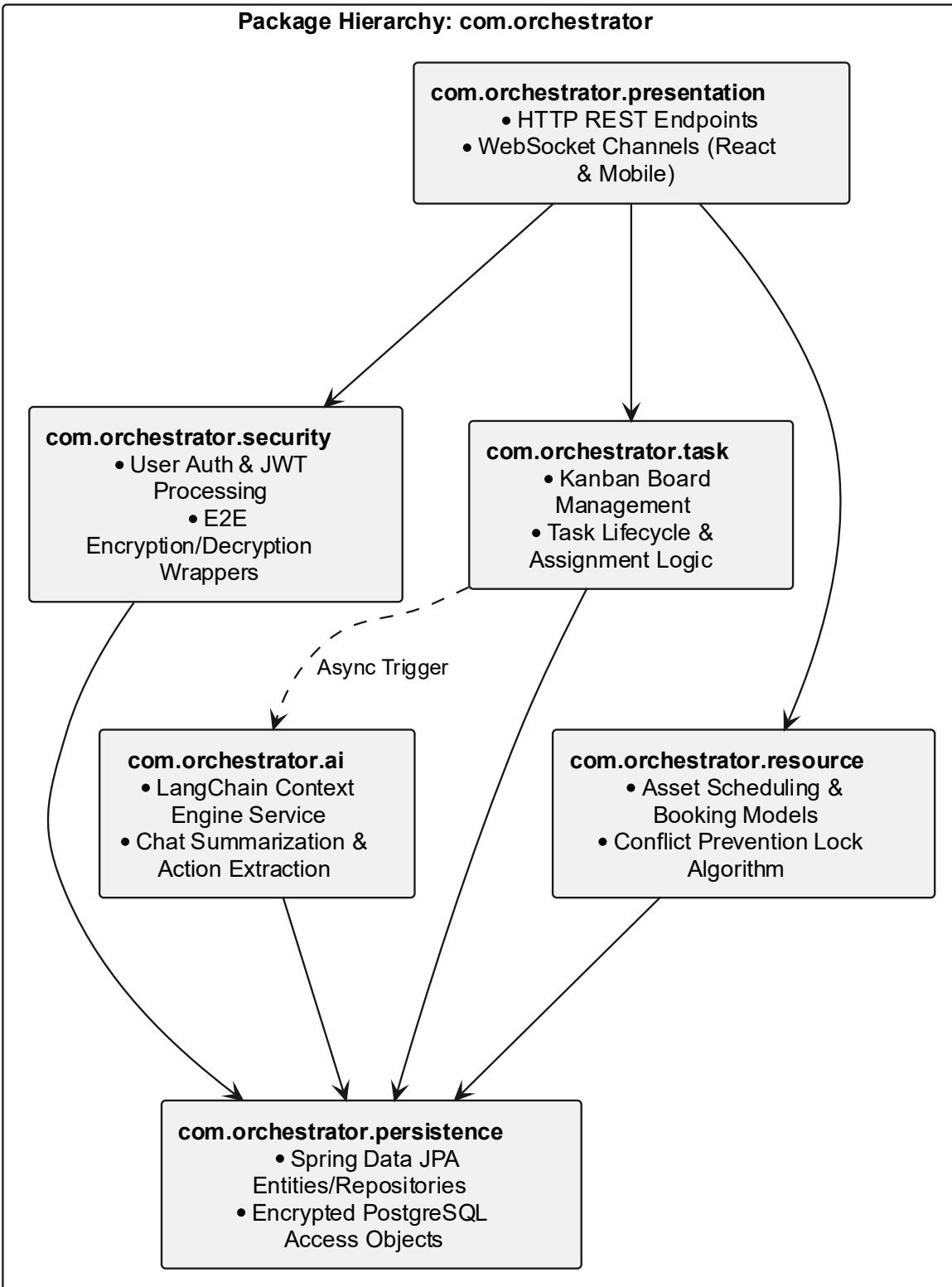
Main flow	1. Context Service pulls text from the Core API. 2. LangChain processes the text to identify key topics and decisions. 3. System formats a summary. 4. Summary is attached to the project documentation.
Successful end/post condition	A concise, automated summary is available for researchers.
Fail end/post condition	Service fails to parse the text contextually.
Extensions	N/A

5. Logical View

The Logical View illustrates the functional decomposition of the Privacy Preserving Collaborative Resource and Task Orchestrator into distinct, decoupled subsystems and software packages. It details how key domain components interact to enforce data privacy, manage shared research assets, and process AI-driven context extraction.

Overview & Package Hierarchy

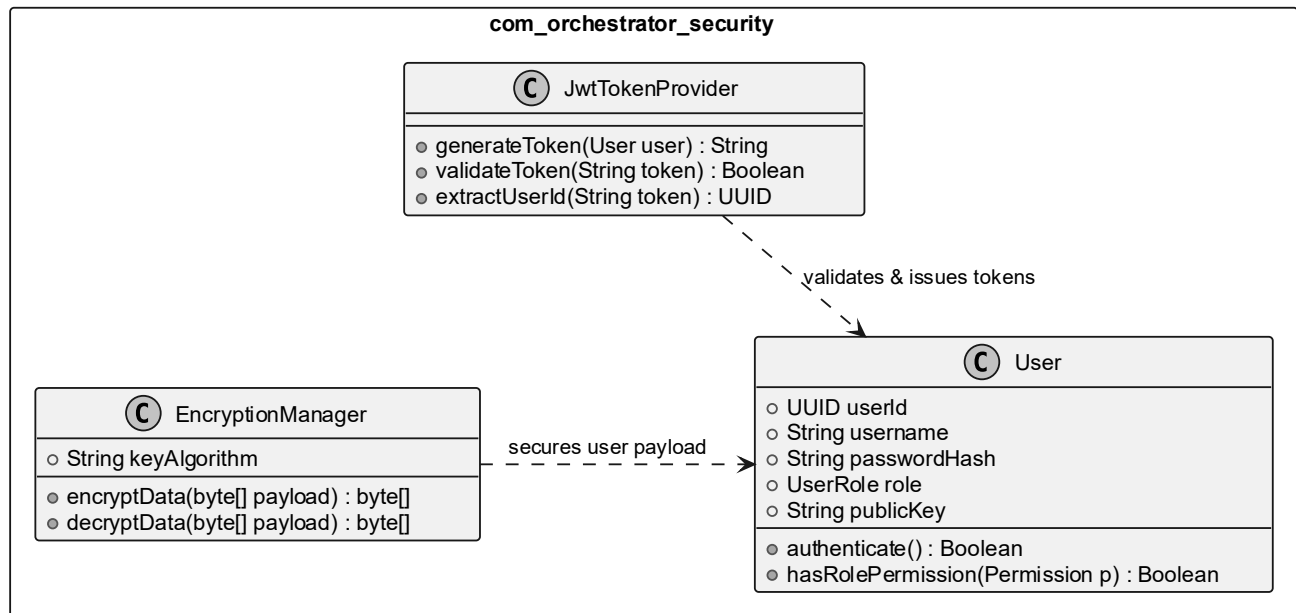
The application follows a layered, microservices capable architectural pattern organized into six major design packages:



Architecturally Significant Design Packages

1. Security Package (com.orchestrator.security)

This package manages user authentication, role-based access control (supporting Institute Admin, PI, Researcher, and Resource Manager roles), token lifecycle management, and encryption interfaces for data sovereignty.



User: Entity representing system actors and their security permissions.

- Responsibilities: Stores user credentials, handles authentication status, and validates user roles (INSTITUTE_ADMIN, PI, RESEARCHER, RESOURCE_MANAGER).
- Attributes: `userId`: UUID, `username`: String, `passwordHash`: String, `role`: UserRole, `publicKey`: String.
- Operations: `authenticate()`, `hasRolePermission(Permission p)`.

JwtTokenProvider: Utility class managing stateless web session tokens.

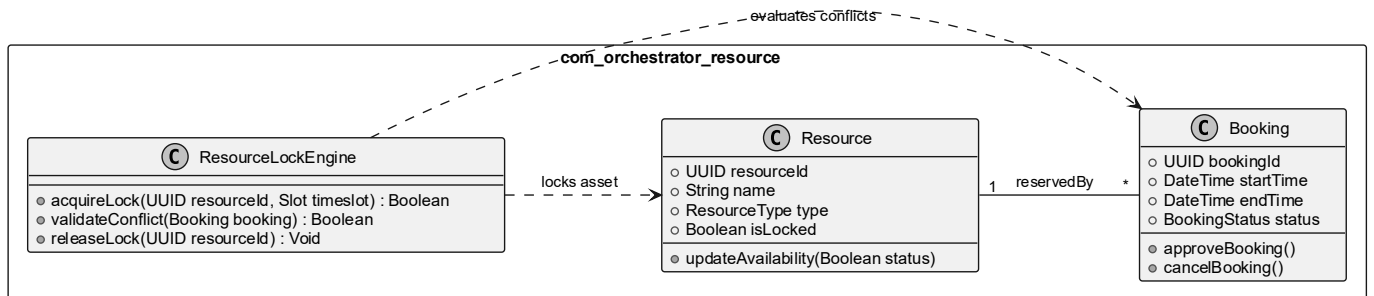
- Responsibilities: Issues signed JSON Web Tokens upon successful user login and validates incoming tokens on API requests.
- Operations: `generateToken(User user)`, `validateToken(String token)`, `extractUserId(String token)`.

EncryptionManager: Service wrapper handling end-to-end encryption mechanics.

- Responsibilities: Encrypts sensitive research payload data prior to database persistence and decrypts payload data for authorized requests.
- Attributes: keyAlgorithm: String.
- Operations: encryptData(byte[] payload), decryptData(byte[] payload).

2. Resource Management Package (com.orchestrator.resource)

This package encapsulates the hardware and asset reservation workflows, preventing double-booking through an atomic resource-locking algorithm.



Resource: Represents a shared research asset available for scheduling, such as lab GPUs, high-performance workstations, or specialized sensors.

- Responsibilities: Maintains physical asset availability state and metadata.
- Attributes: resourceId: UUID, name: String, type: ResourceType, isLocked: Boolean.
- Operations: updateAvailability(Boolean status).

Booking: Entity tracking a scheduled reservation slot created by a researcher.

- Responsibilities: Holds start/end timestamps and approval status for hardware usage requests.
- Attributes: bookingId: UUID, startTime: DateTime, endTime: DateTime, status: BookingStatus.

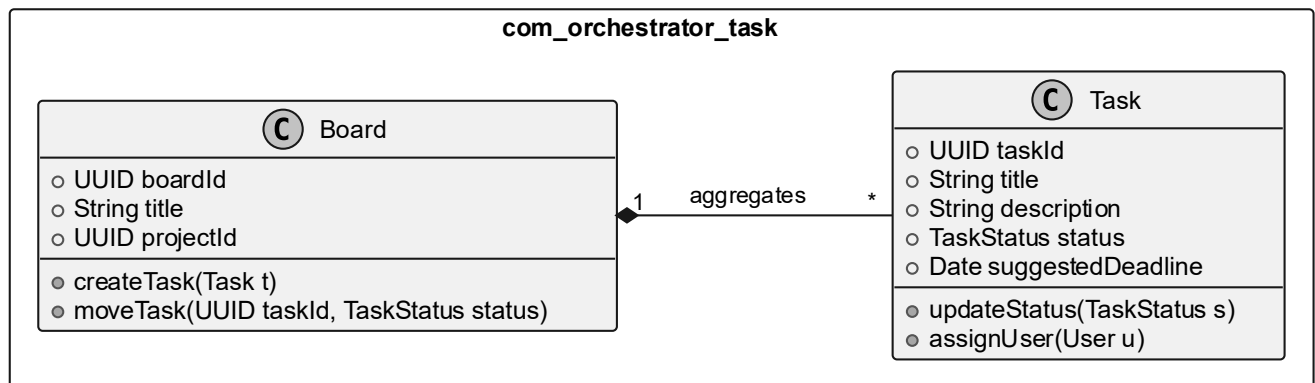
- Operations: approveBooking(), cancelBooking().

ResourceLockEngine: Architectural utility enforcing concurrent booking constraints.

- Responsibilities: Evaluates reservation timeslots to eliminate overlapping bookings and executes atomic locks on shared resources.
- Operations: acquireLock(UUID resourceId, Slot timeslot), validateConflict(Booking newBooking), releaseLock(UUID resourceId).

3. Task Management Package (com.orchestrator.task)

This package governs the visual task boards, task lifecycles, interactive Kanban cards, and user assignments across projects.



Board: Aggregator component representing an interactive Kanban task board for a research project.

- Responsibilities: Groups related tasks, controls board columns, and manages card transitions across stages.
- Attributes: boardId: UUID, title: String, projectId: UUID.
- Operations: createTask(Task t), moveTask(UUID taskId, TaskStatus status).

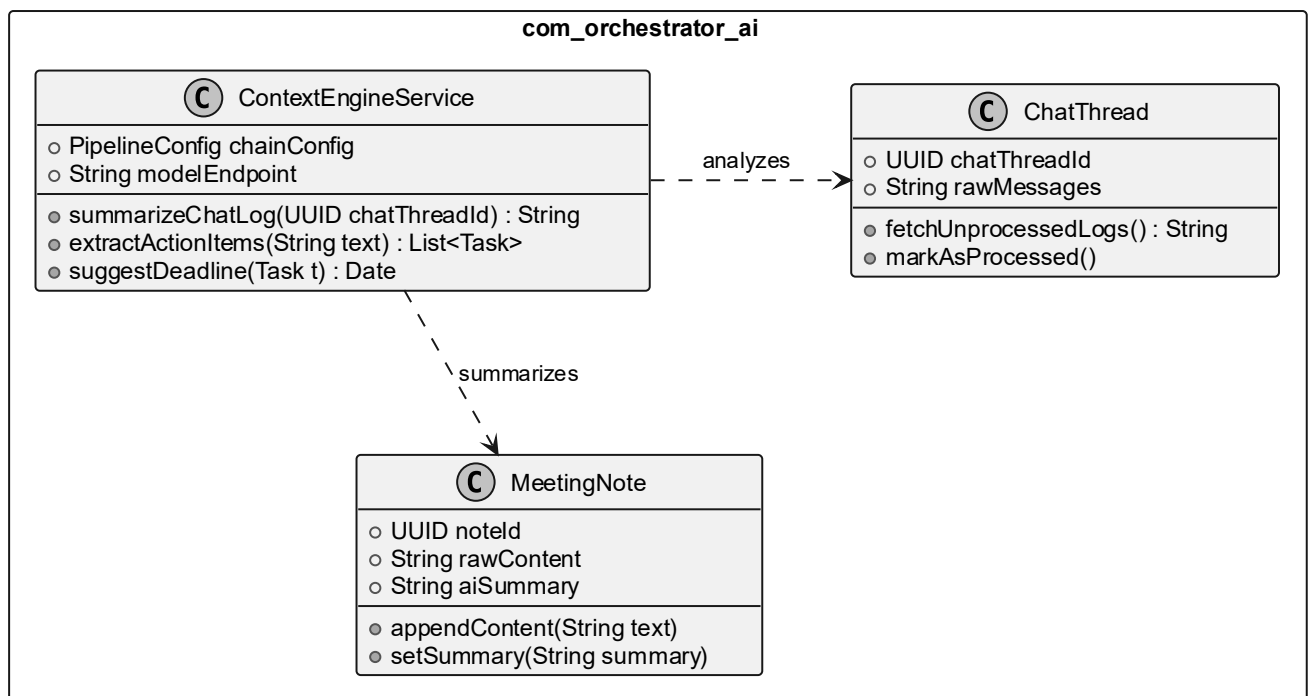
Task: Entity representing individual work items, experiment milestones, or research actions.

- Responsibilities: Tracks task progress status, team member assignments, and deadline suggestions.

- Attributes: taskId: UUID, title: String, description: String, status: TaskStatus, suggestedDeadline: Date.
- Operations: updateStatus(TaskStatus s), assignUser(User u).

4. AI Context Package (com.orchestrator.ai)

This package provides automated background processing using LangChain to extract action items, generate meeting summaries, and suggest task deadlines from unstructured conversations.



ContextEngineService: Backend microservice interface wrapping LangChain processing pipelines.

- Responsibilities: Integrates LLM chains to analyze chat logs asynchronously, parse action items, and compute proposed deadlines.
- Attributes: chainConfig: PipelineConfig, modelEndpoint: String.

- Operations: summarizeChatLog(UUID chatThreadId), extractActionItems(String text), suggestDeadline(Task t).

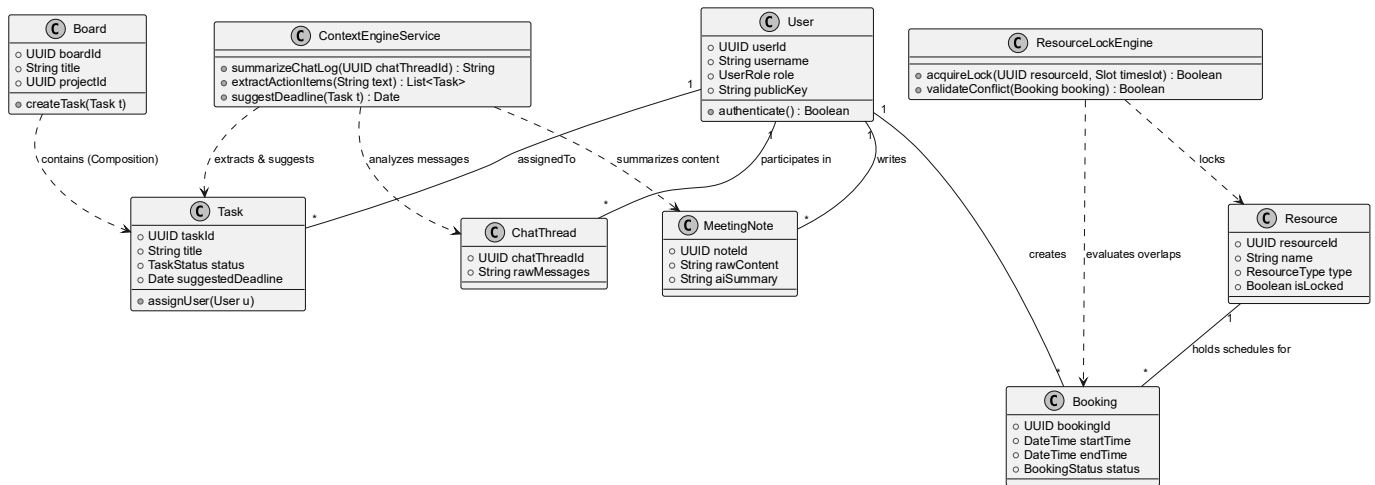
MeetingNote: Holds rich-text collaborative document records and generated meeting summaries.

- Responsibilities: Stores raw collaborative user text input and displays machine-generated meeting summaries.
- Attributes: noteId: UUID, rawContent: String, aiSummary: String.
- Operations: appendContent(String text), setSummary(String summary).

ChatThread: Data container holding unstructured communication histories.

- Responsibilities: Streams ongoing discussion text to the AI Context Engine for analysis.
- Attributes: chatThreadId: UUID, rawMessages: String.
- Operations: fetchUnprocessedLogs(), markAsProcessed().

Overall Architecturally Significant Class Diagram



Class Diagram Description

This holistic class diagram captures the structural foundation of the application, bridging the gaps between identity, task execution, physical resource allocation, and AI automation.

Key Relationships and Responsibilities:

- **User as the Central Actor:** The User class (acting as a Researcher, PI, or Admin) is the anchor of the system. Users are assigned to Tasks, they create Bookings for physical lab equipment, and they generate the text data inside MeetingNotes and ChatThreads.
- **Task Aggregation (Composition):** The Board class has a strict composition relationship with Task. The board serves as the interactive Kanban workspace that aggregates all tasks belonging to a specific research project.
- **Resource Scheduling & Concurrency:** A physical lab asset (Resource) can have multiple timeslots reserved (Booking). Because double-booking hardware is a critical failure point in academic labs, the ResourceLockEngine sits between them as a utility to evaluate new bookings and apply atomic locks to the resource.
- **AI-Driven Task Generation:** The ContextEngineService operates as an independent backend processor. It establishes a dependency relationship with the communication classes (ChatThread and MeetingNote). By processing these text sources through LangChain, it automatically generates structured Task objects (extracting action items and suggesting deadlines) without requiring manual entry from the users.

6. Process View

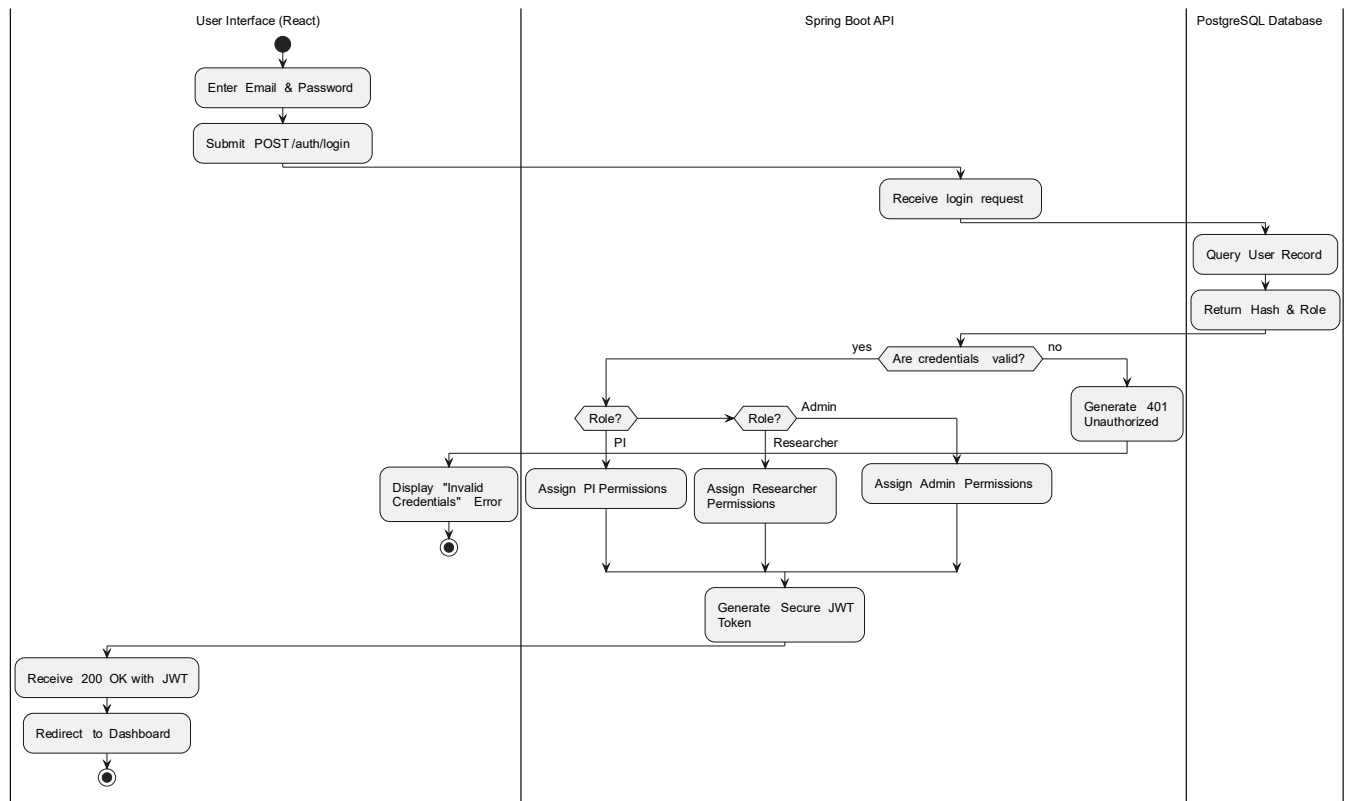
This section details the dynamic behavior of the system, illustrating how the containerized microservices communicate at runtime to execute the primary workflows. It focuses specifically on concurrency, synchronization, and the step-by-step logic of human-system interactions.

6.1 Activity Diagrams

The following activity diagrams illustrate the step-by-step logical flows, decision points, and parallel processes for the core features of the system.

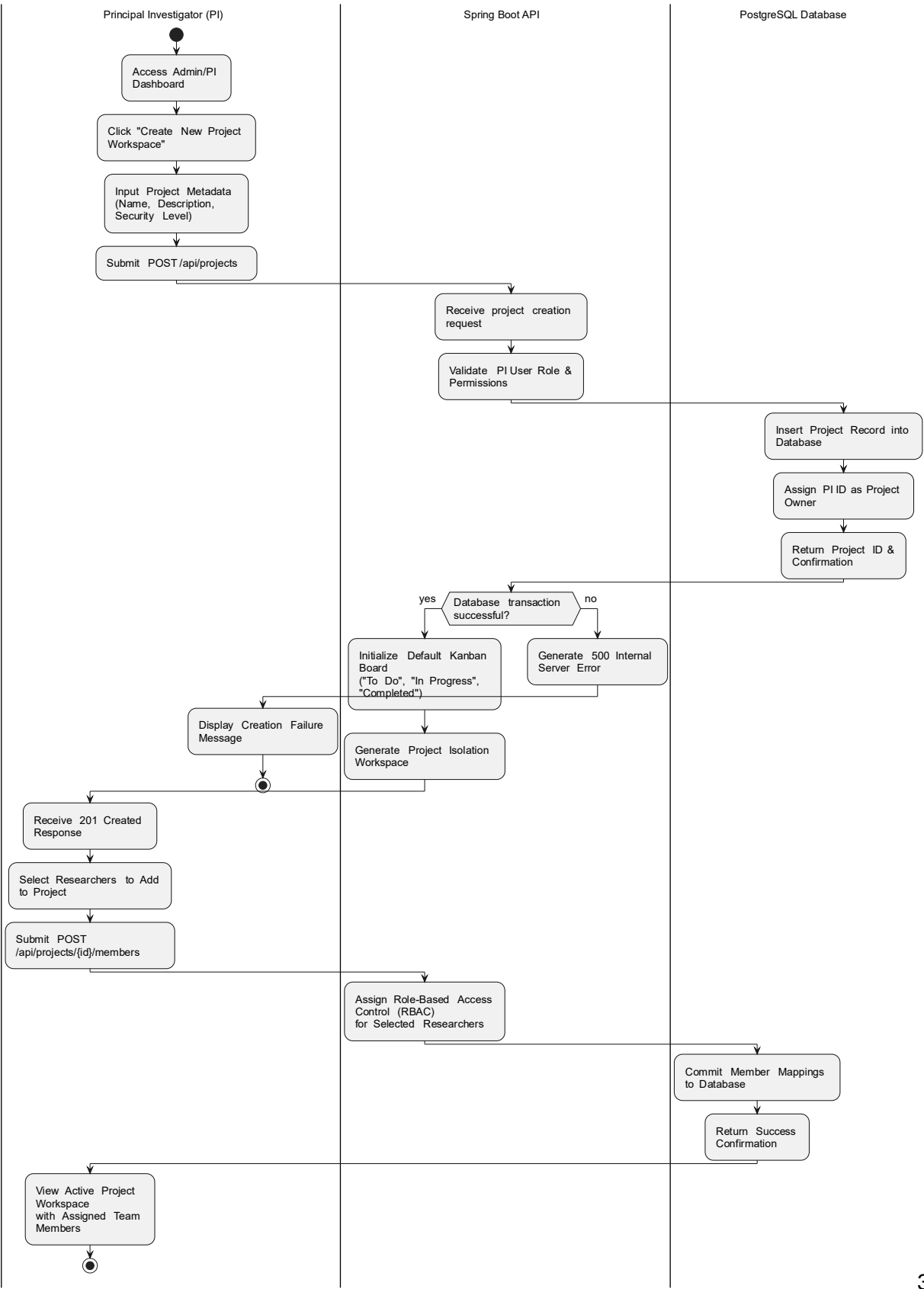
6.1.1 User Authentication and Authorization Process

This diagram illustrates the login workflow. It details the decision paths when a user submits their credentials, how the Spring Boot API validates the password hash against the PostgreSQL database, and the branching logic for generating a secure JWT session token based on the user's role (Principal Investigator, Researcher, or Admin) versus returning an authentication error.



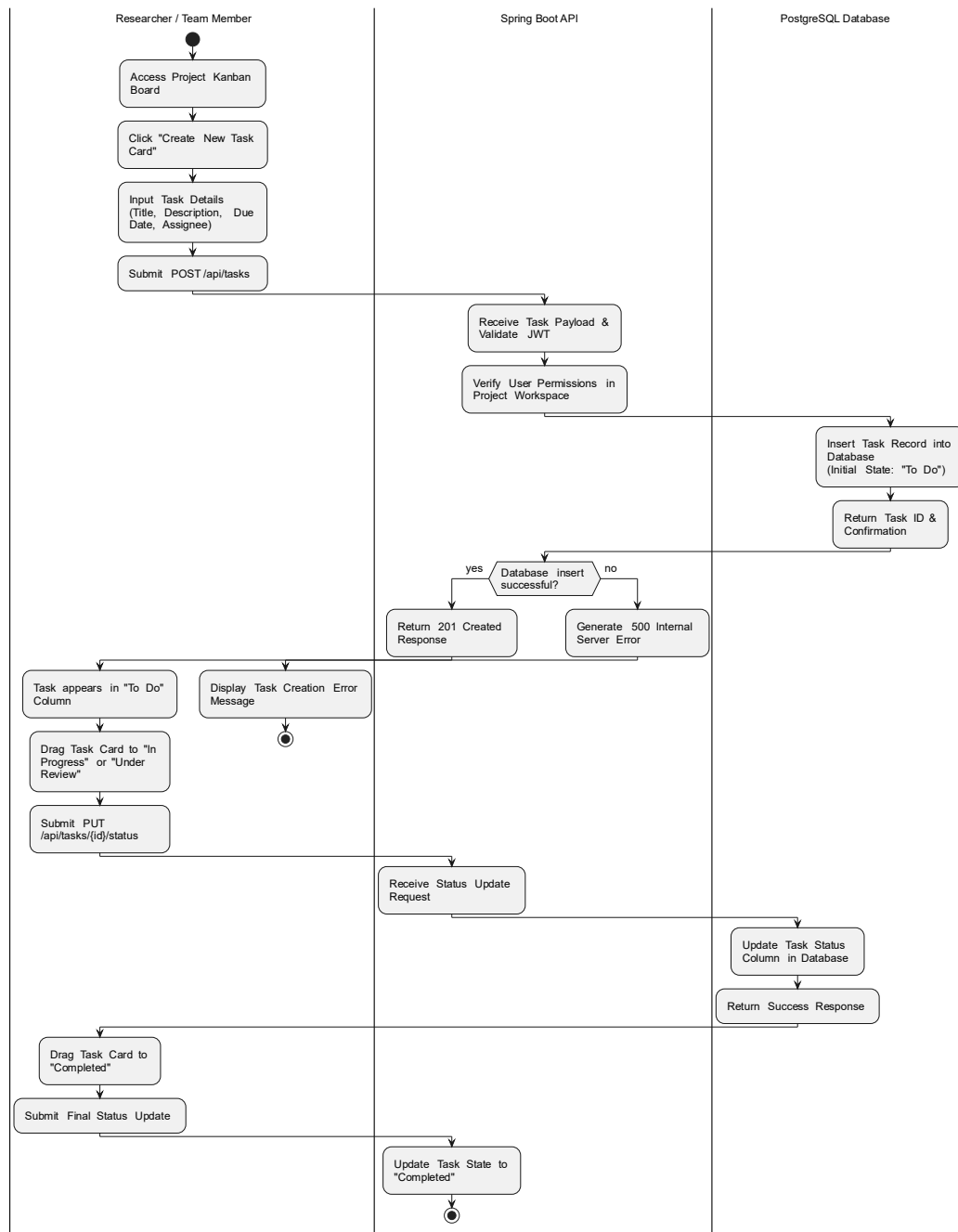
6.1.2 Project Workspace Setup

This workflow maps the steps taken by a Principal Investigator (PI) to initialize a new research project. It shows the sequential flow of defining project parameters, searching the system directory for researchers, assigning specific Role-Based Access Control (RBAC) permissions, and the system generating the isolated project workspace.



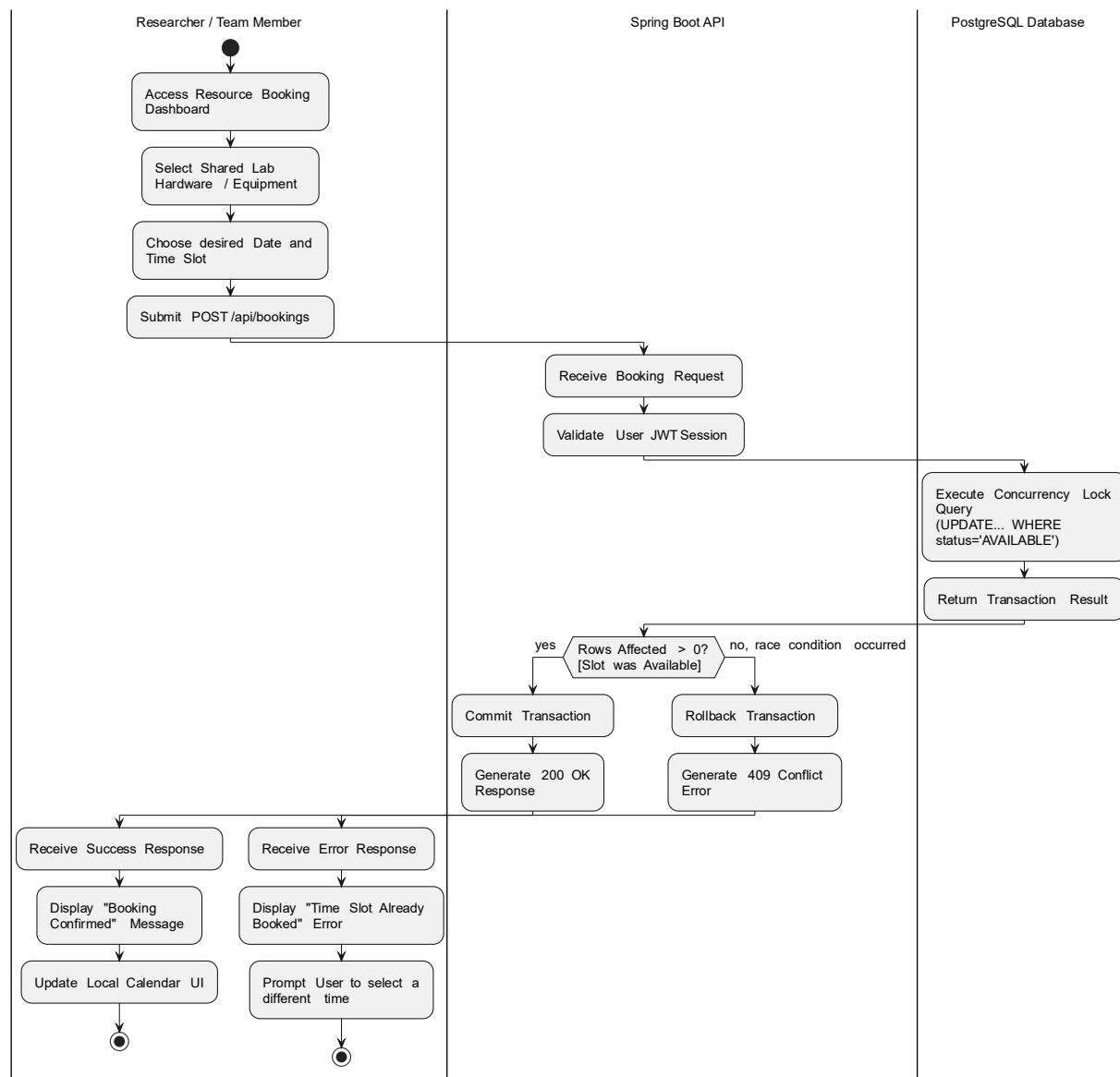
6.1.3 Kanban Task Lifecycle Flow

This diagram maps the lifecycle of a task card within the interactive Kanban board. It illustrates the human logic of creating a task, assigning a deadline, and the system-level decision nodes triggered when a user drags the card across columns (To-Do, In Progress, Review, Completed), updating the database state at each stage.



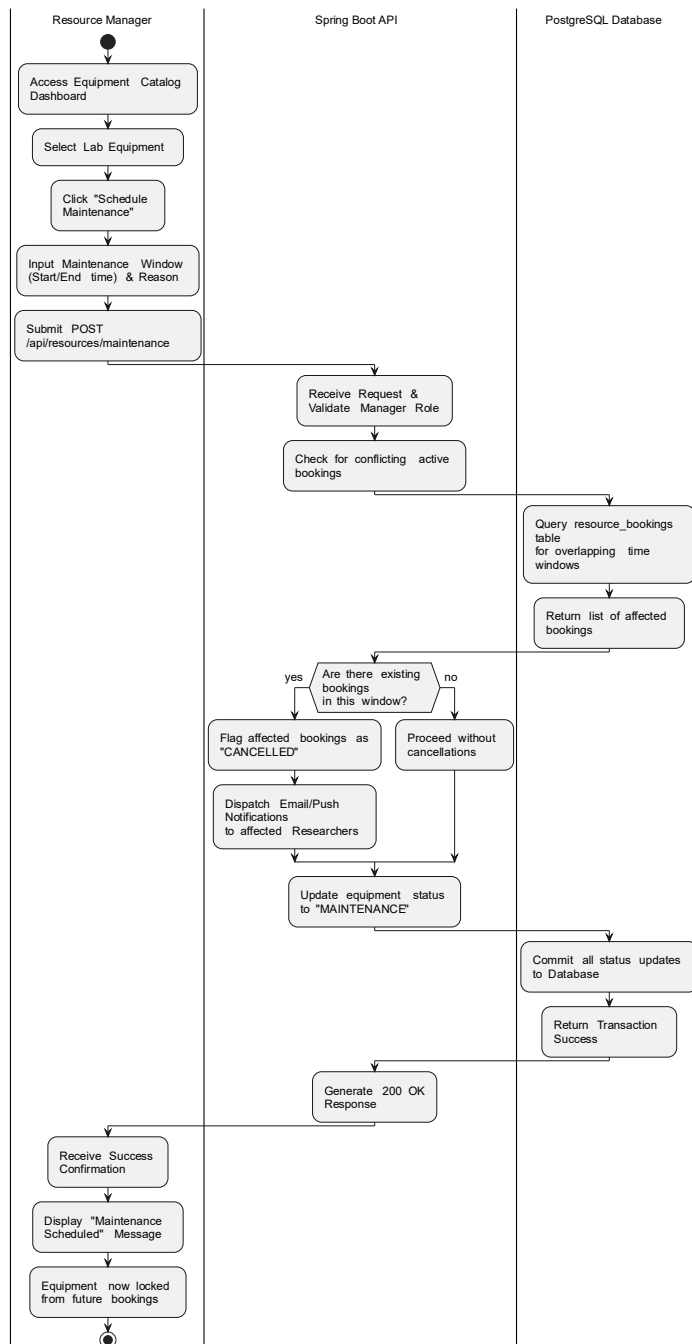
6.1.4 Resource Booking and Conflict Resolution

This diagram details the logic for booking shared laboratory hardware. It highlights the critical decision node where the system checks the database for overlapping time slots. If the slot is available, the system locks it and confirms; if a race condition occurs and the slot is taken, the system routes the user to an error state to select a new time.



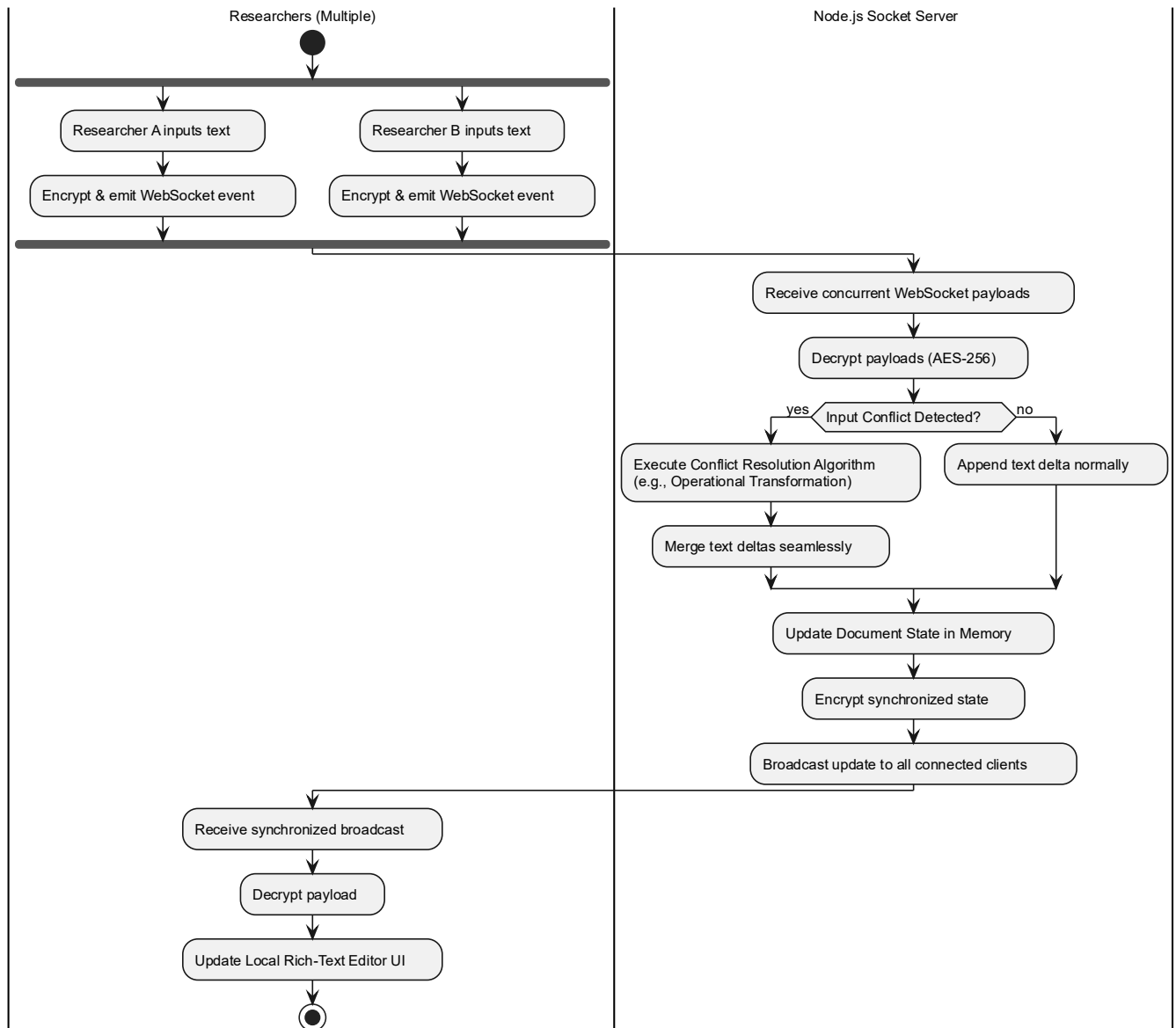
6.1.5 Resource Maintenance Scheduling

This workflow illustrates the administrative process of taking lab equipment offline. It maps how a Resource Manager sets a maintenance window, and how the system automatically checks for existing bookings within that window, cancels them, and triggers email/push notifications to the affected researchers.



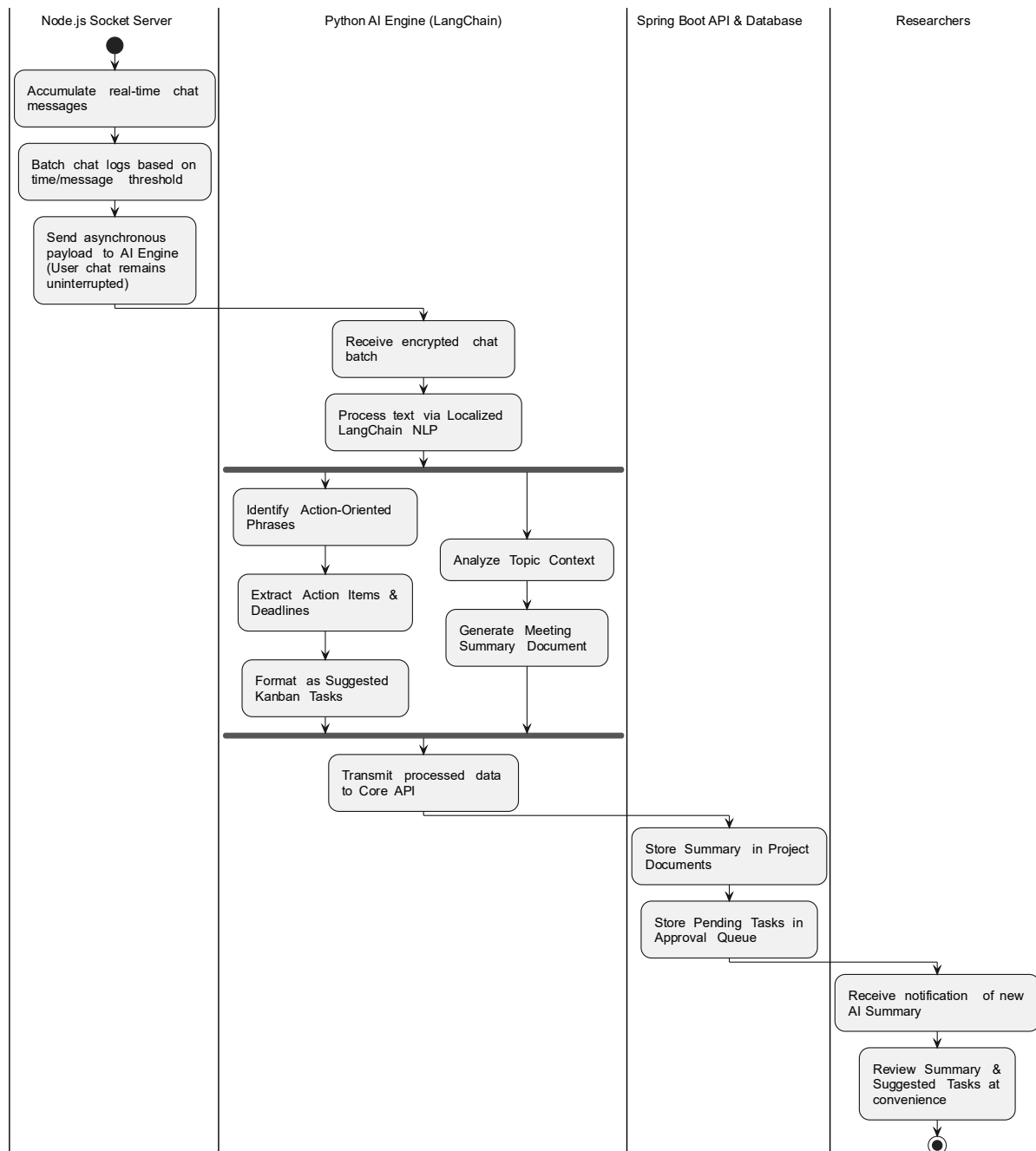
6.1.6 Real-Time Document Synchronization

This diagram shows the parallel processing logic required for collaborative text editing. It illustrates how the Node.js WebSocket server handles simultaneous keystrokes from multiple researchers, resolves input conflicts in real-time, and broadcasts the synchronized document state back to all connected clients without data loss.



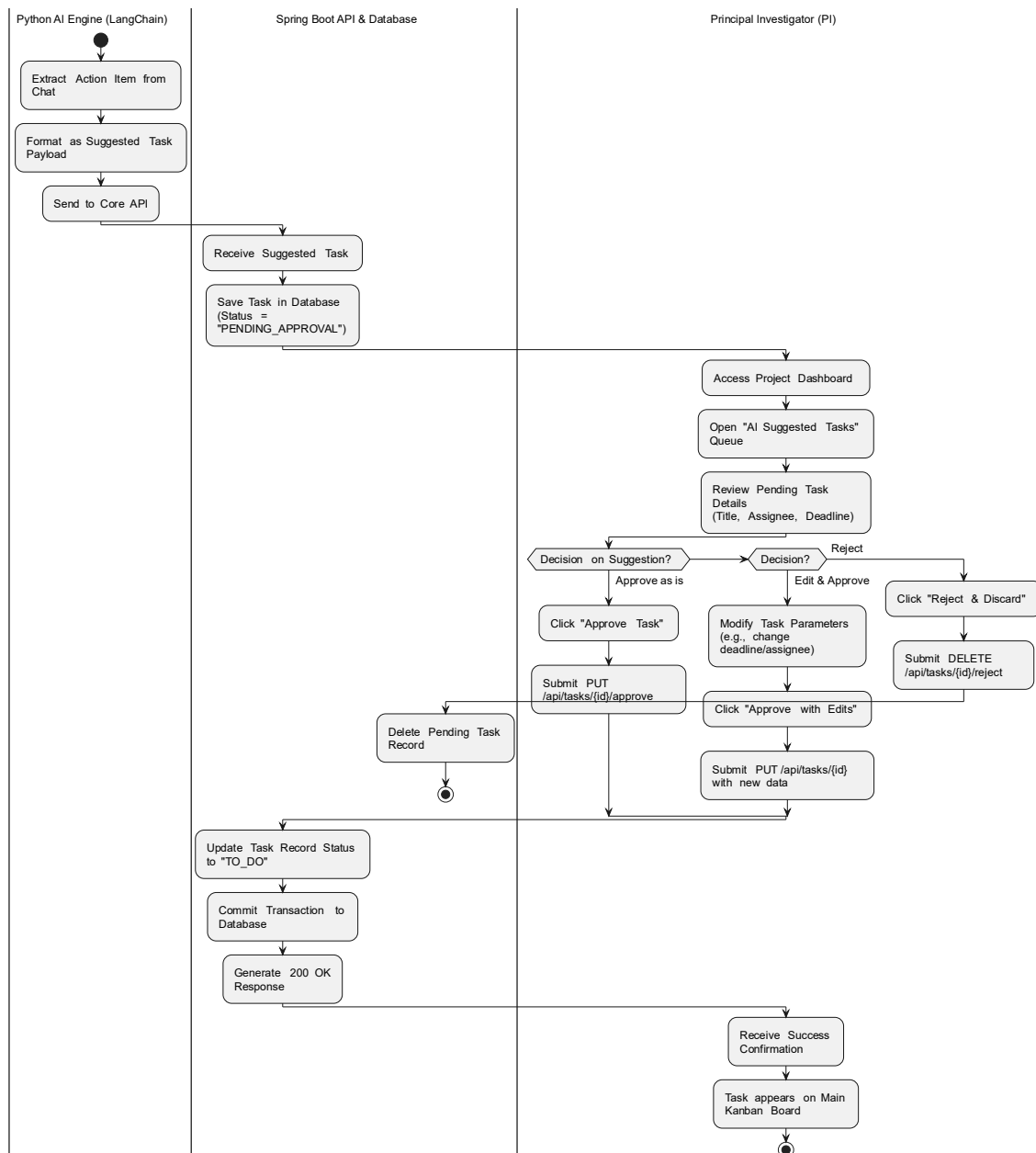
6.1.7 AI Context Processing Workflow

The Automated Context Engine operates asynchronously. This diagram illustrates how chat logs are securely batched and sent to the localized Python engine, which then autonomously utilizes LangChain NLP to extract action items and generate summary documents without interrupting the users' real-time chat experience.



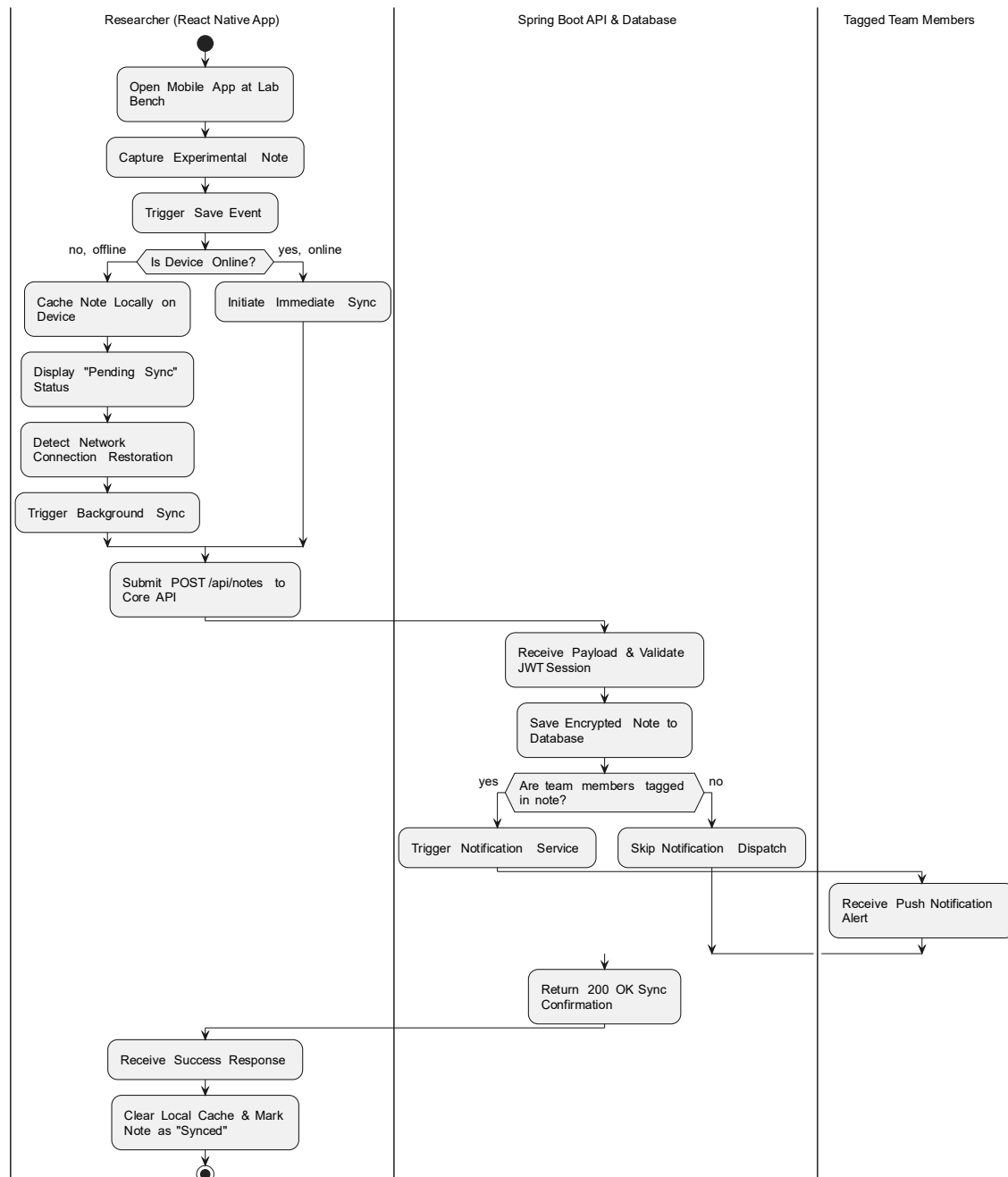
6.1.8 AI Task Approval Process

This diagram maps the human-in-the-loop workflow for AI-generated suggestions. Once the AI extracts a potential action item from a meeting, it is placed in a pending state. The diagram shows the PI's decision paths to either approve the task (moving it to the Kanban board), edit the parameters, or reject and discard the suggestion.



6.1.9 Mobile Note Capture and Sync

This workflow illustrates the offline/online logic of the React Native mobile application. It shows a researcher capturing an experimental note at the lab bench. If the device lacks network connectivity, the note is cached locally; once a connection is detected, the system syncs the data to the Spring Boot API and dispatches push notifications to tagged team members.

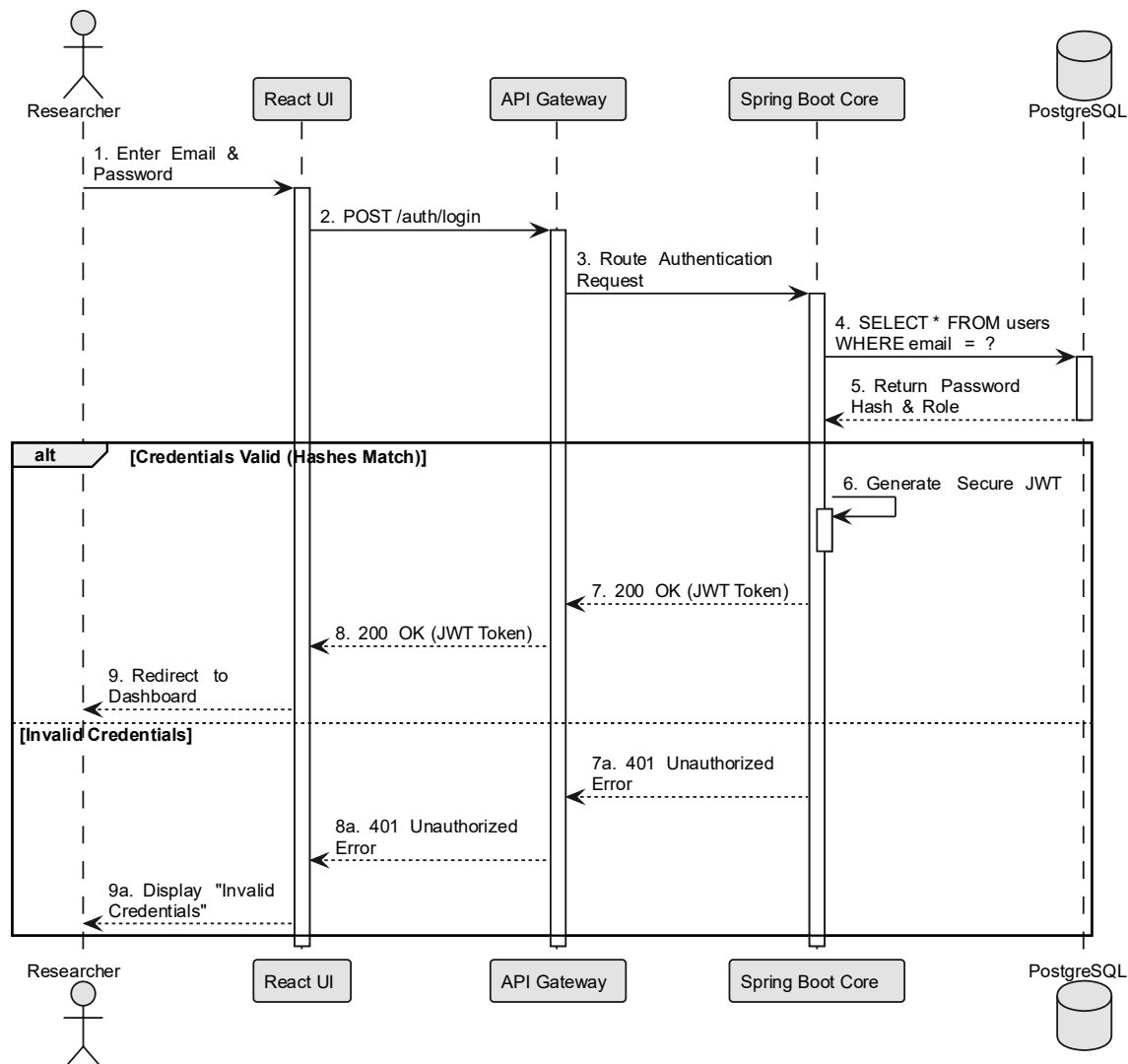


6.2 Sequence Diagrams

The following sequence diagrams illustrate the exact time-ordered flow of messages, API calls, and data transfers between the system's external actors (users) and internal microservices (React UI, API Gateway, Spring Boot Core, Node.js, Python AI, and PostgreSQL).

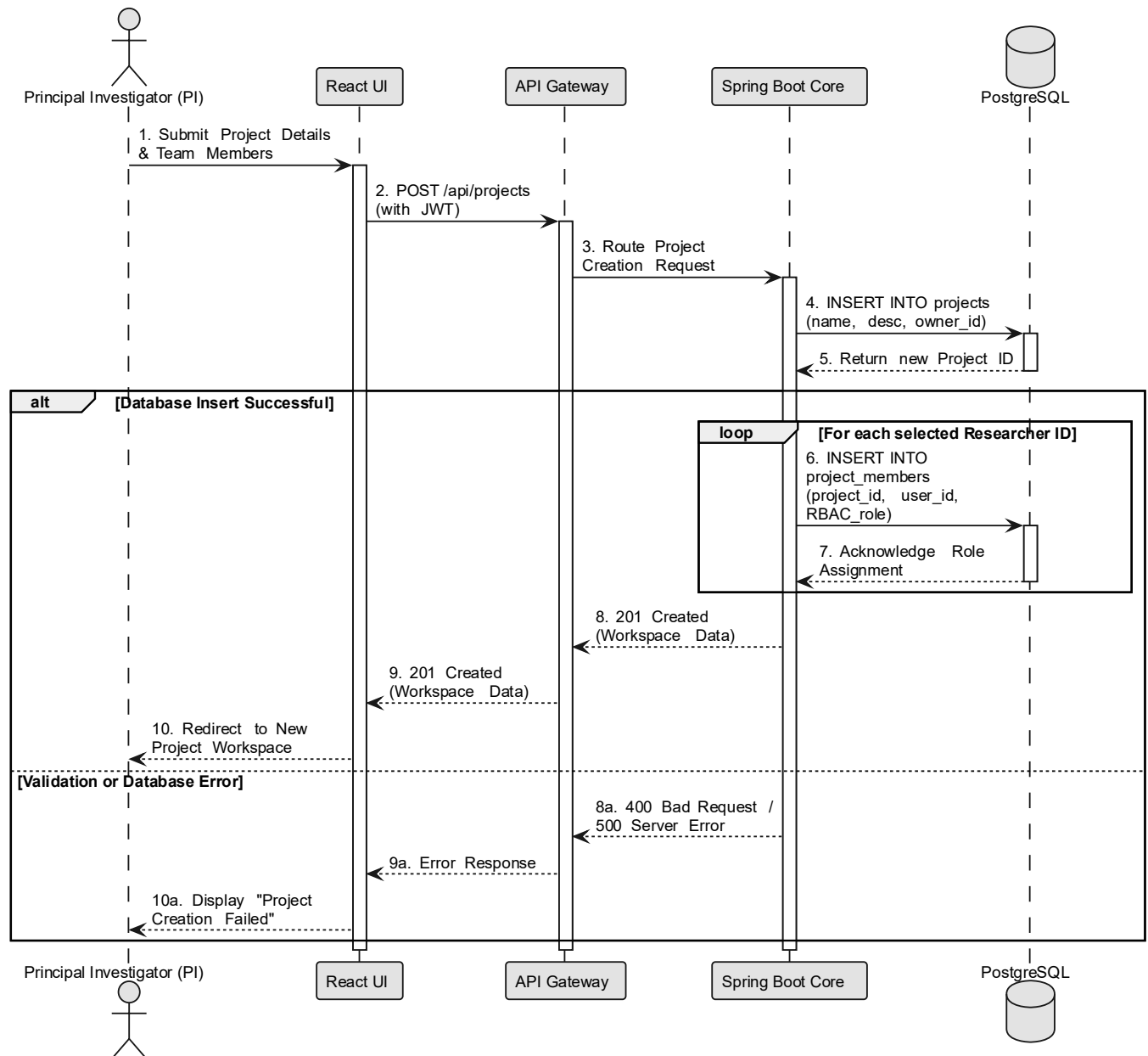
6.2.1 User Authentication & JWT Generation

This diagram demonstrates the secure login sequence. It details the Researcher sending credentials via the React frontend, the Spring Boot Core API querying the PostgreSQL database to validate the password hash, and the subsequent generation and return of a secure JSON Web Token (JWT) for session management.



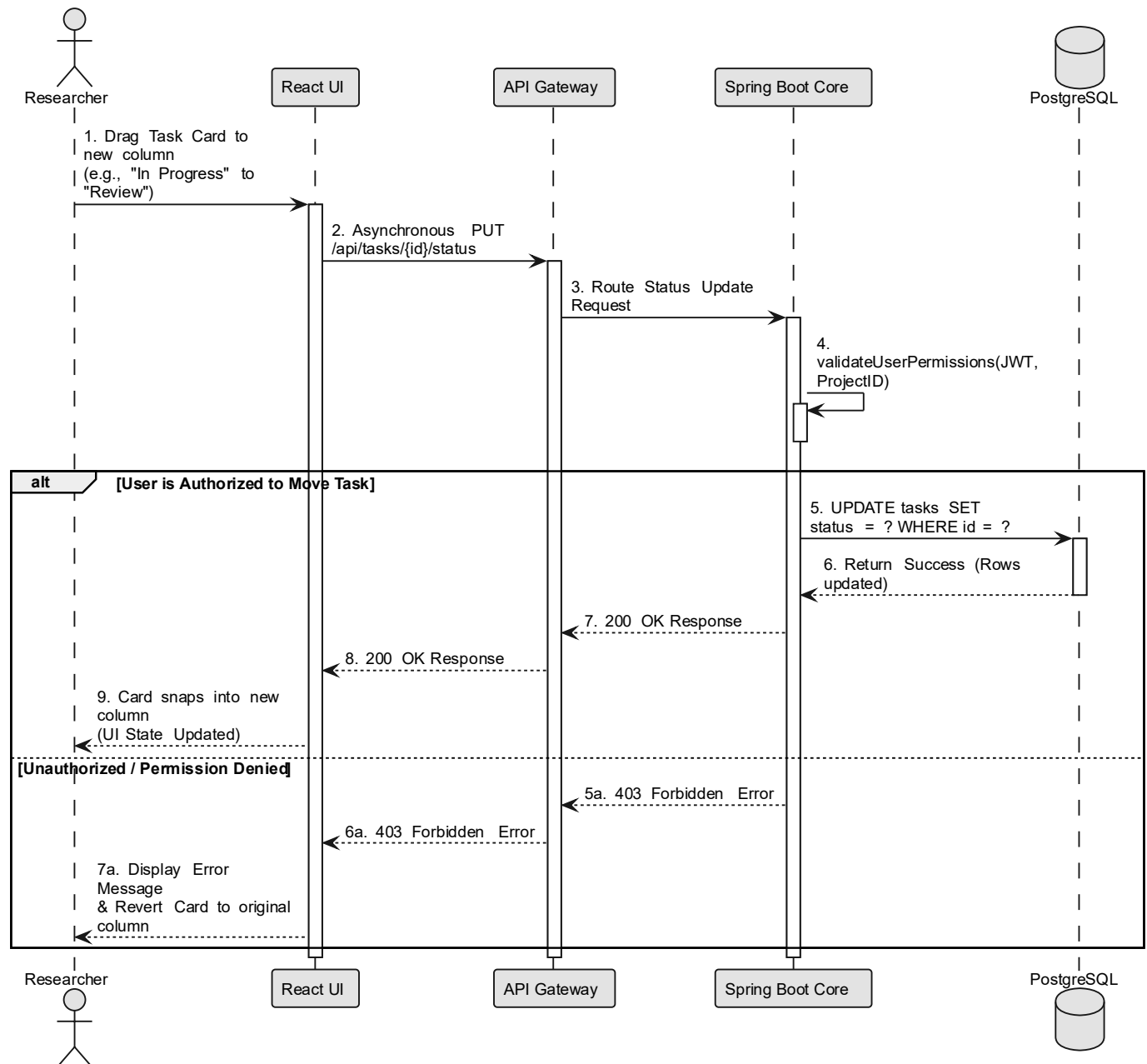
6.2.2 Project Creation & Role Assignment

This sequence maps the API interactions when a Principal Investigator (PI) initializes a workspace. It shows the UI sending a POST request to the API Gateway, the Spring Boot service creating the project record in the database, and the subsequent mapping of Researcher IDs to specific Role-Based Access Control (RBAC) profiles.



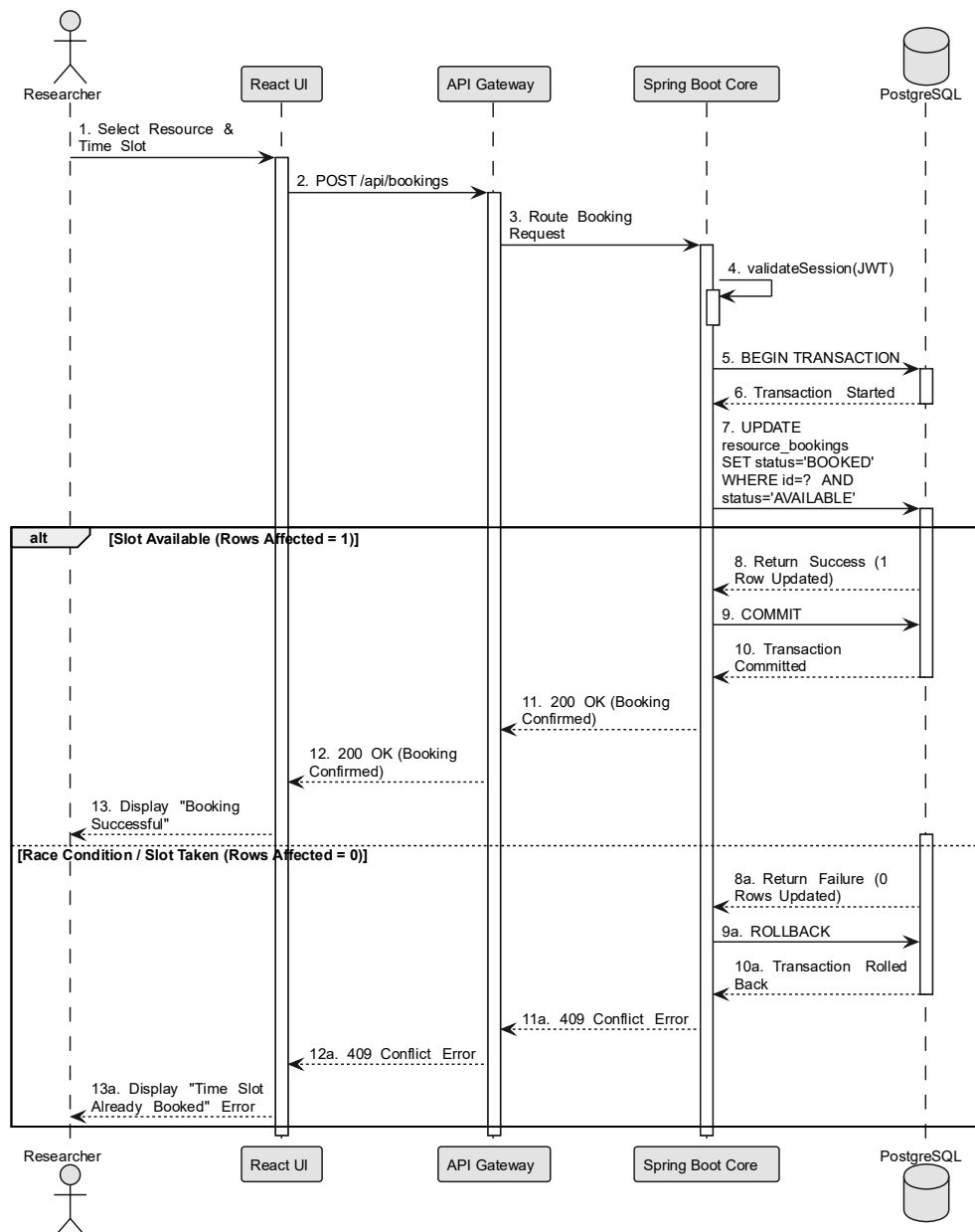
6.2.3 Kanban Task Management

This diagram illustrates the data flow of interacting with the Kanban board. It shows the sequence of a user dragging a task card, the frontend dispatching an asynchronous PUT request to the API, and the database updating the task's state (e.g., from "In Progress" to "Review") before returning a 200 OK success response.



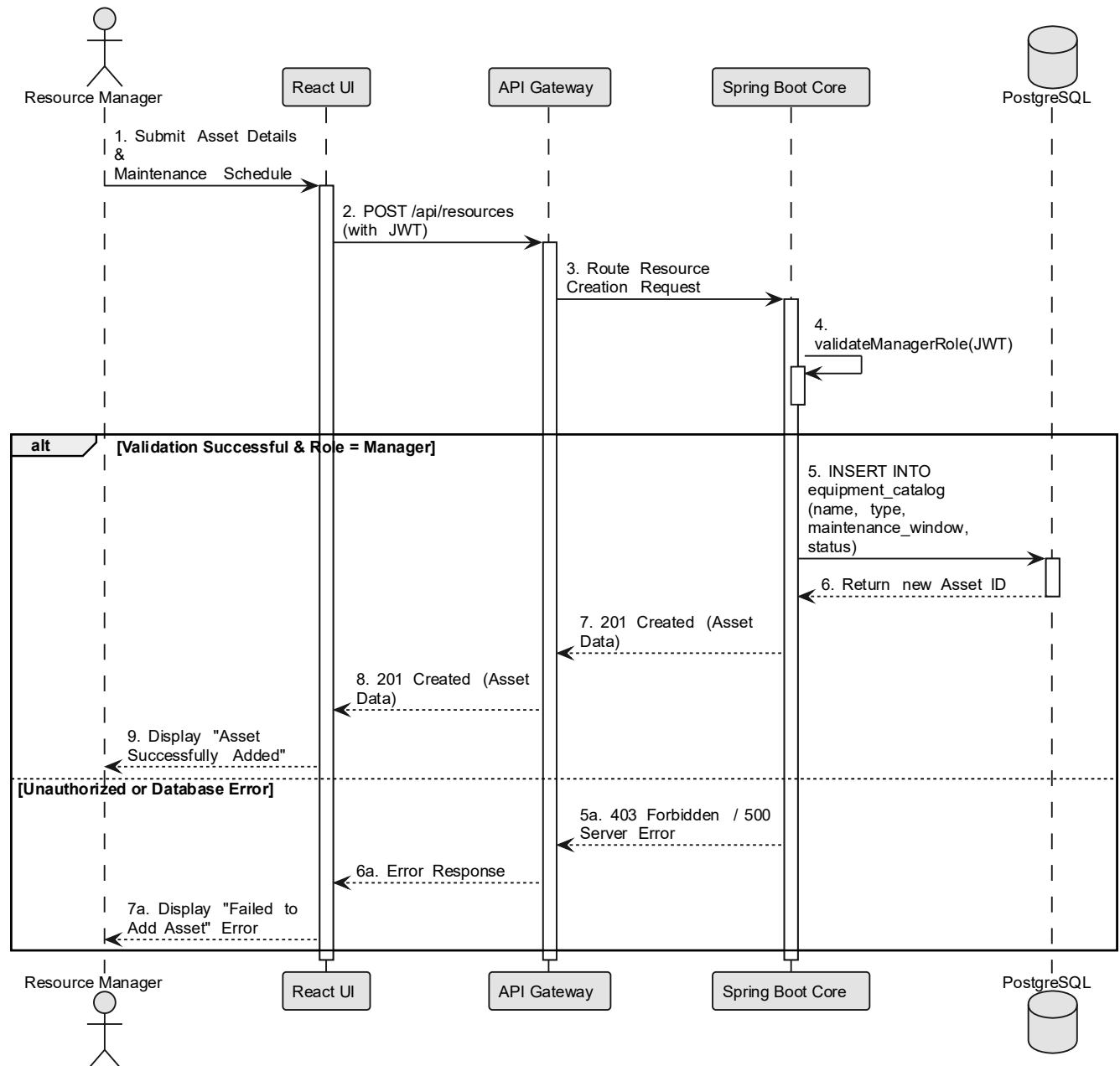
6.2.4 Resource Booking with Concurrency Lock

This sequence details the strict database-level locking mechanism used to prevent double-booking of lab hardware. It maps the race condition where two users request the same time slot simultaneously, showing how PostgreSQL locks the row, grants a 200 OK to the first transaction, and returns a 409 Conflict error to the second.



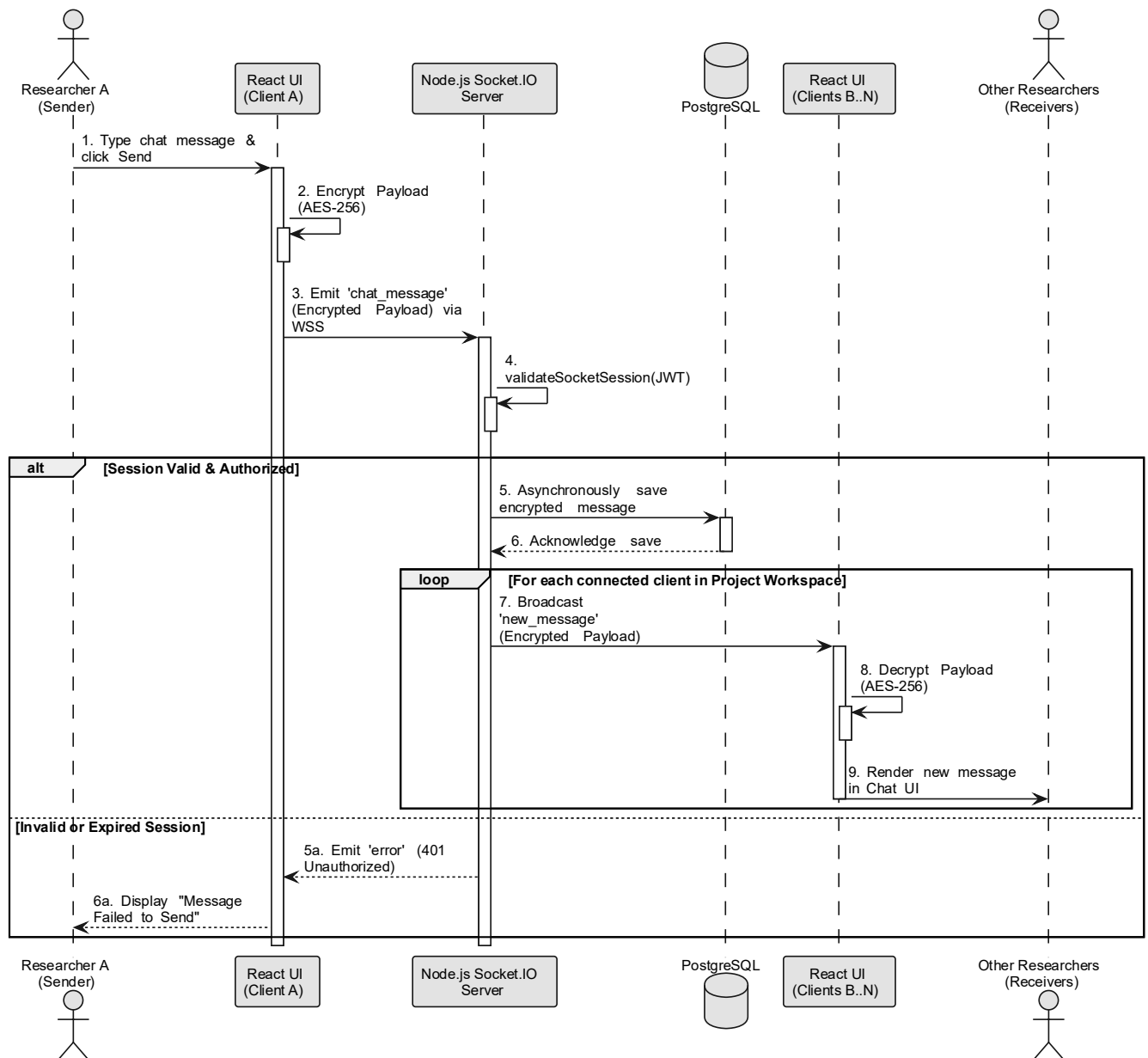
6.2.5 Equipment Catalog Management

This diagram shows the administrative data flow for resource management. It outlines the sequence of a Resource Manager adding a new lab asset to the system, defining its maintenance schedule, and the Core API committing these parameters to the database.



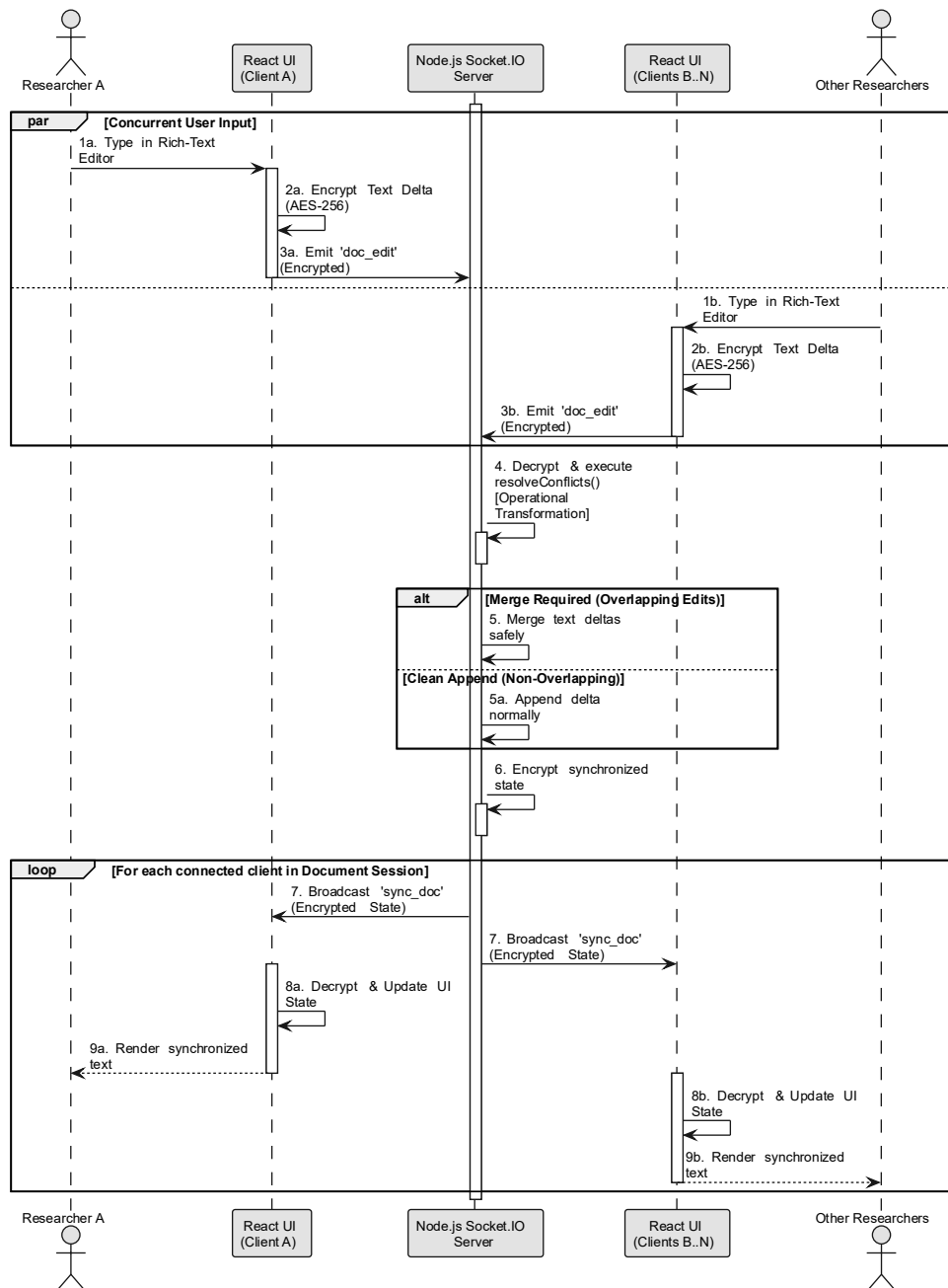
6.2.6 Secure Real-Time Chat (WebSockets)

This sequence illustrates the low-latency communication pipeline. It maps a researcher sending a message, the AES-256 encryption process, the transmission to the Node.js Socket.IO server, and the immediate broadcast of the encrypted payload to all other connected clients in the research workspace.



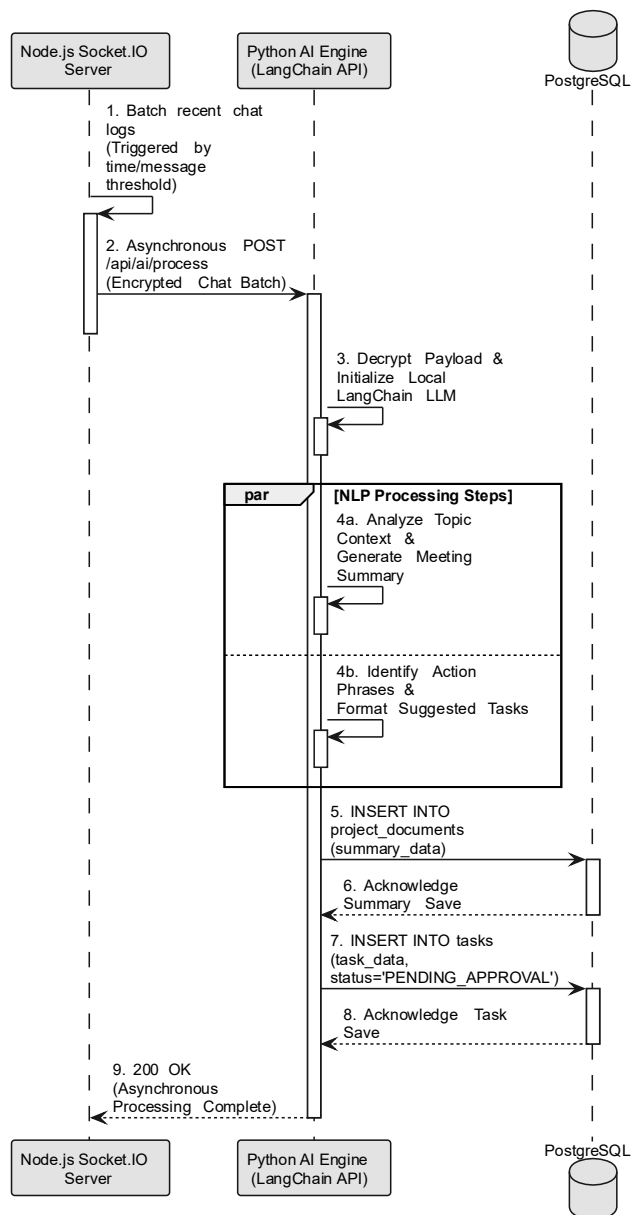
6.2.7 Synchronous Collaborative Document Editing

This diagram outlines the real-time data synchronization for the rich-text editor. It shows multiple researchers transmitting keystrokes simultaneously to the Node.js server, which processes the inputs using conflict-resolution logic before broadcasting the synchronized document state back to the clients.



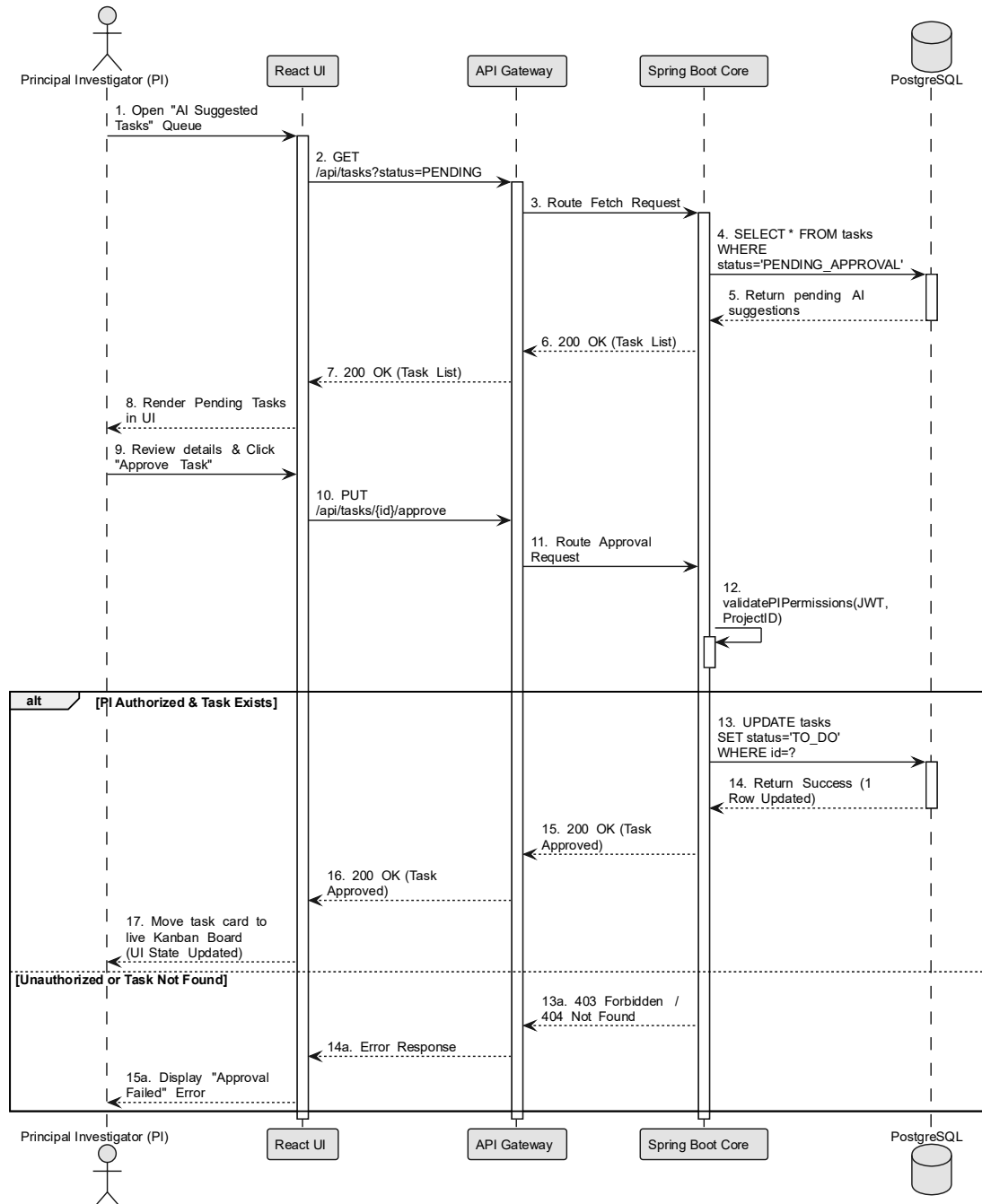
6.2.8 Background AI Chat Summarization

This sequence details the asynchronous interaction between the chat server and the AI engine. It shows the Node.js server batching recent chat logs, transmitting them to the localized Python (LangChain) API, the NLP processing steps, and the Python service saving the generated summary directly to the PostgreSQL database.



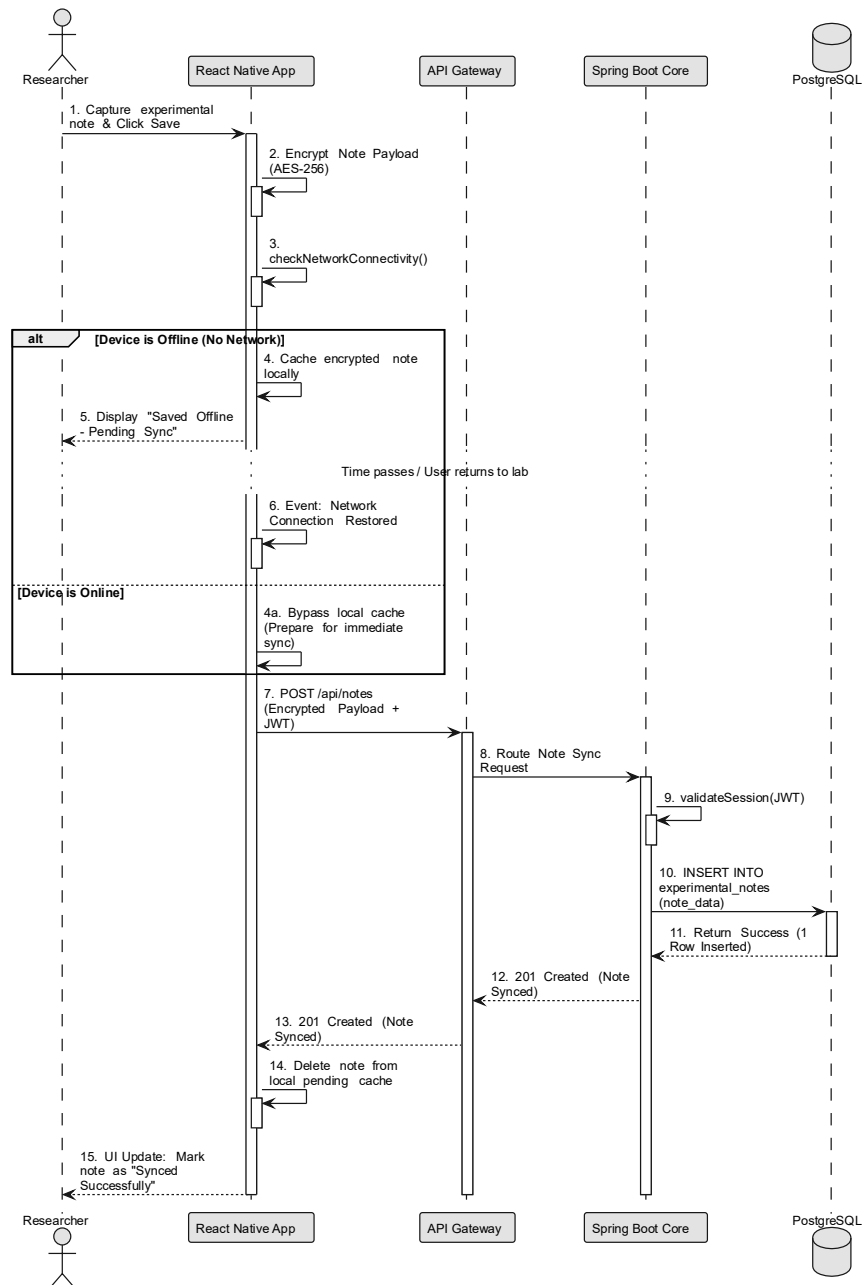
6.2.9 AI Suggested Task Approval

This diagram maps the interaction between the user and the AI-generated task queue. It shows the PI retrieving pending AI suggestions from the database, approving a specific action item via the UI, and the Spring Boot API converting that suggestion into a live Kanban task.



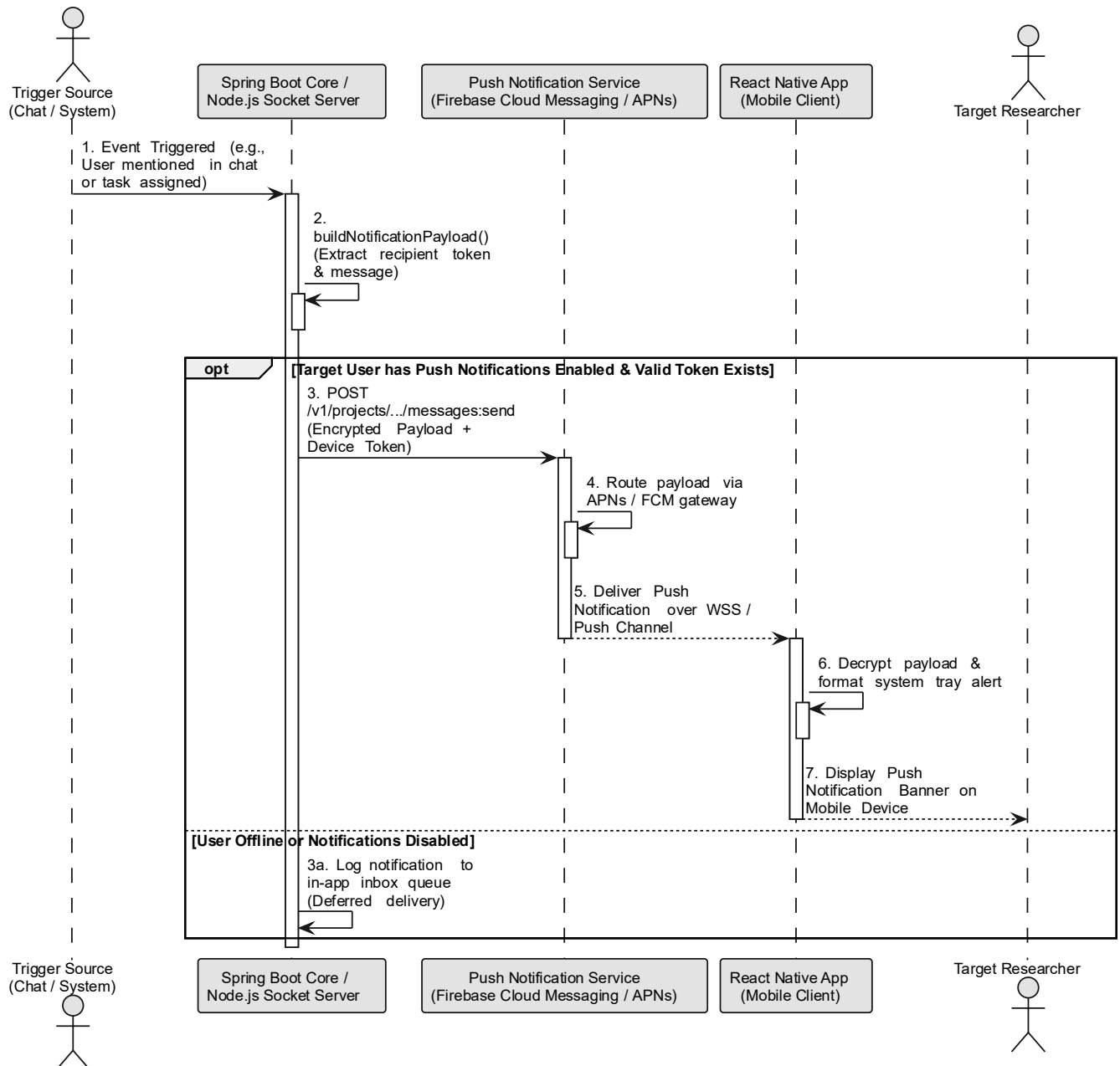
6.2.10 Mobile Quick-Capture Note Sync

This sequence illustrates the mobile synchronization pipeline. It details the React Native app caching an experimental note locally, detecting network availability, and securely POSTing the payload through the API Gateway to be stored in the core database.



6.2.11 Mobile Push Notification Dispatch

This diagram shows the event-driven sequence for mobile alerts. It maps the trigger event (e.g., a user being tagged in a chat), the Node.js/Spring Boot service dispatching a notification payload to the push service, and the delivery of the alert to the researcher's iOS/Android device.



7. Deployment View

This section describes the physical hardware configuration and the mapping of software components to those hardware nodes. To ensure the strict "Privacy-First" and data sovereignty requirements, the system is designed to be self-hosted on a private laboratory server utilizing a containerized microservices architecture.

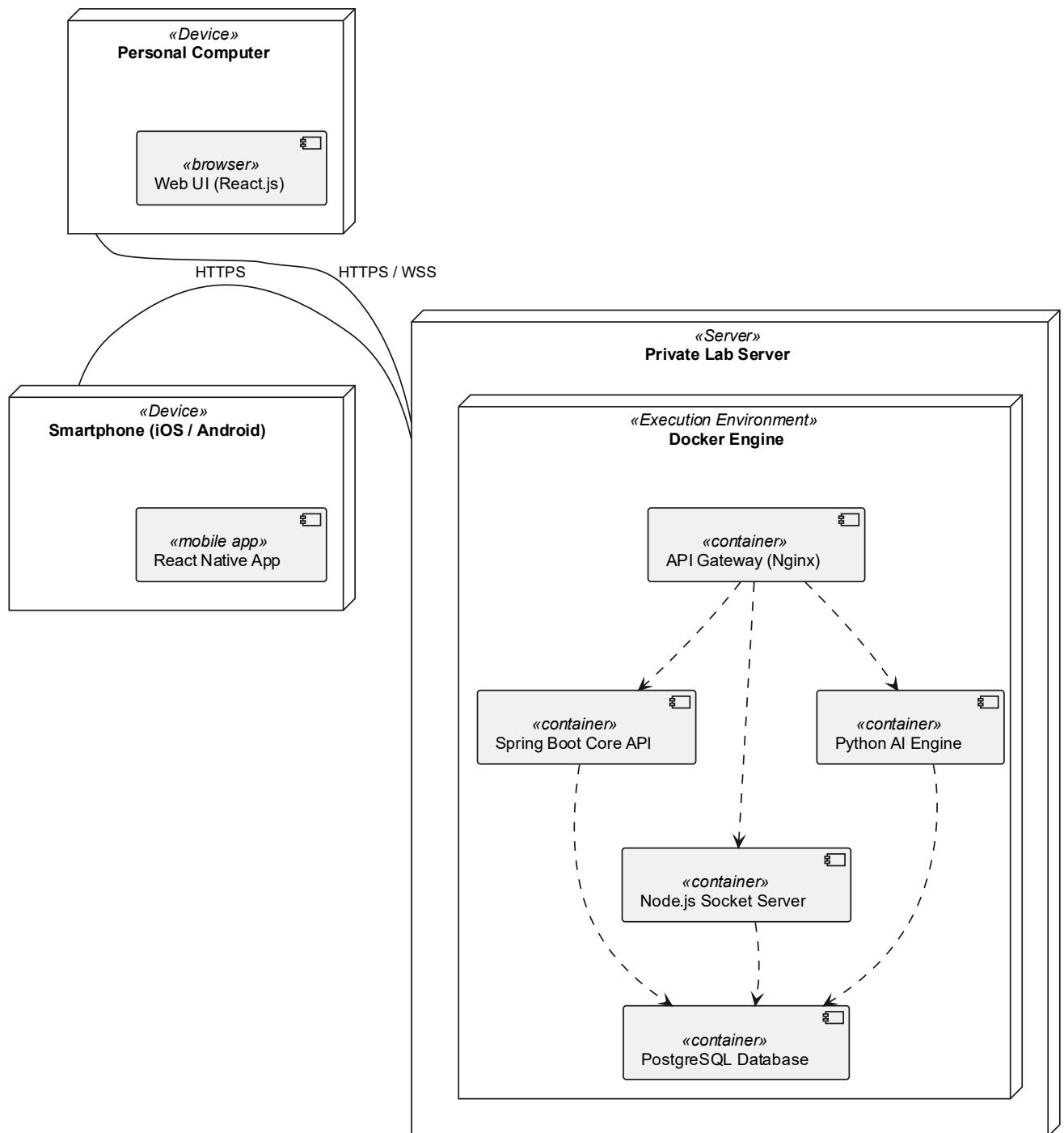
7.1 Physical Nodes and Connections

- **Client Devices:** Researchers access the platform via standard personal computers (using a web browser for the React.js dashboard) or smartphones (using the React Native companion app).
- **Network:** Communication between client devices and the server occurs over a secure Local Area Network (LAN) or a Virtual Private Network (VPN) using encrypted HTTPS and WSS (WebSocket Secure) protocols.
- **Private Lab Server:** A dedicated physical server or private cloud instance (minimum 8GB RAM) located strictly within the institution's controlled infrastructure. It runs the Docker Engine to host the system's isolated microservices.

7.2 Mapping of Software to Hardware (Containerization)

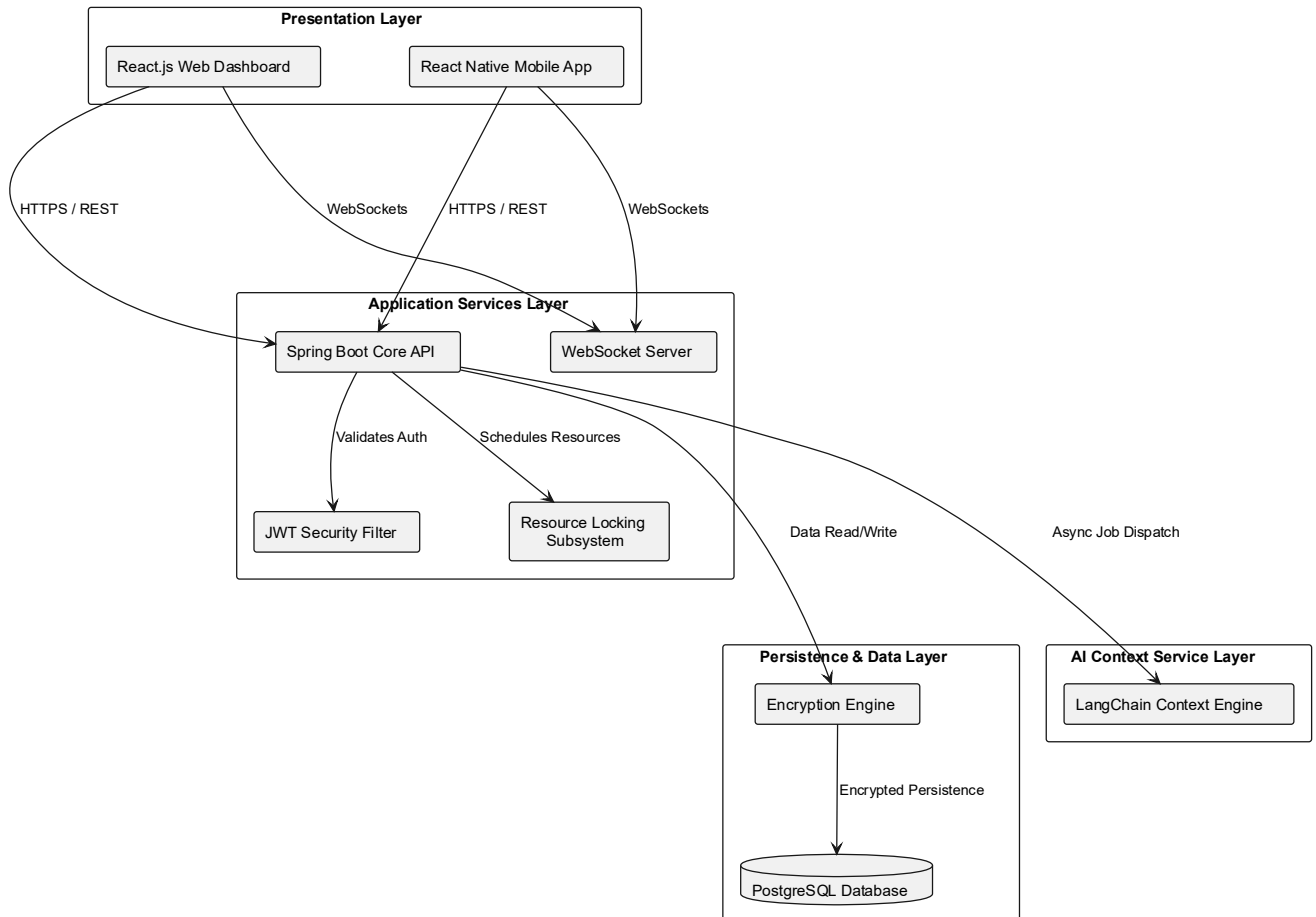
The Private Lab Server hosts several independent Docker containers managed via Docker Compose. By isolating these services, the system ensures graceful degradation and high maintainability:

- **API Gateway / Web Container:** Acts as a reverse proxy (e.g., Nginx) to serve the React.js frontend and route incoming API traffic to the appropriate backend service.
- **Core API Container:** Executes the Java Spring Boot application responsible for user authentication, business logic, and database transactions.
- **Real-Time Socket Container:** Executes the Node.js Socket.io server to handle low-latency collaborative document editing and live chat messaging.
- **AI Context Engine Container:** Executes the Python (FastAPI and LangChain) service for localized natural language processing, ensuring chat logs are never sent to external AI servers.
- **Database Container:** Hosts the PostgreSQL relational database, storing all AES-256 encrypted system data securely on a mounted local hardware volume.



8. Implementation View

The implementation model decomposes the Privacy Preserving Collaborative Resource and Task Orchestrator into four isolated layers containerized within a single-tenant Docker environment. This architecture decouples user interface clients, core business logic, AI orchestration, and persistent storage to ensure strict data sovereignty and fault isolation on local research lab hardware.



Overview

Layers Definition & Rules

- **Presentation Layer:** Houses the web and mobile interfaces. It handles user interaction, Kanban rendering, resource scheduling views, and mobile push notifications.
- **Application Services Layer:** Enforces business constraints, manages JWT authentication, handles resource reservation locks, and maintains WebSocket channels for real-time collaboration.
- **AI Context Service Layer:** Operates as an independent microservice housing the LangChain backend to process discussion threads, summarize meeting notes, and extract action items.
- **Persistence & Data Layer:** Manages persistent storage via Dockerized PostgreSQL and handles data encryption at rest and in transit.

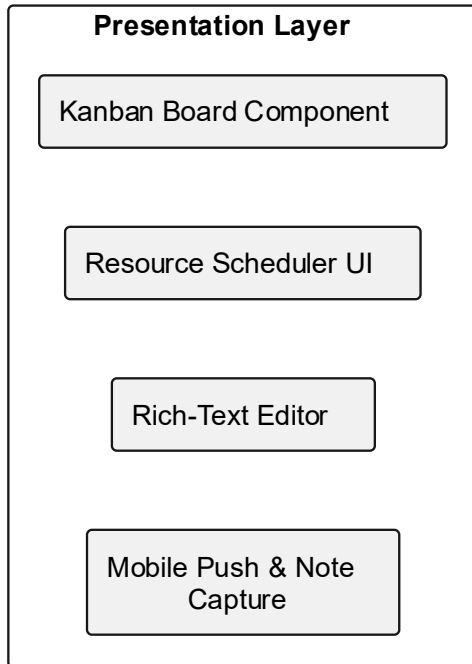
Layer Rules & Boundaries

- **Strict Top-Down Calls:** Upper layers may call lower layers, but lower layers must never import or depend directly on upper layers.
- **API Boundary:** Direct database access from client applications is prohibited; all data access must be validated by the Application Services Layer.
- **Asynchronous Execution Boundary:** The AI Context Engine interacts with the application layer asynchronously to prevent blocking core transactional APIs during heavy processing.

Layers

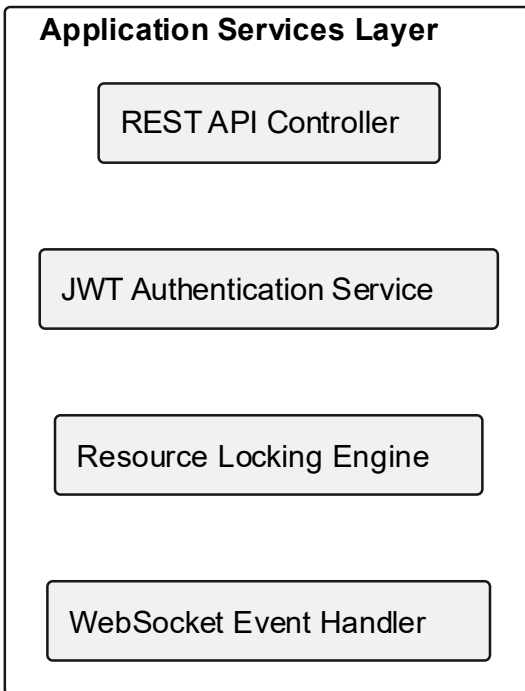
Layer 1: Presentation Layer

- **Subsystems:**
 - **Web Application Subsystem:** Built with React.js, featuring interactive Kanban boards, resource scheduling dashboards, and rich-text meeting documentation editors.
 - **Mobile Companion Subsystem:** Built with React Native, providing real-time push notification handling and quick-capture note tools



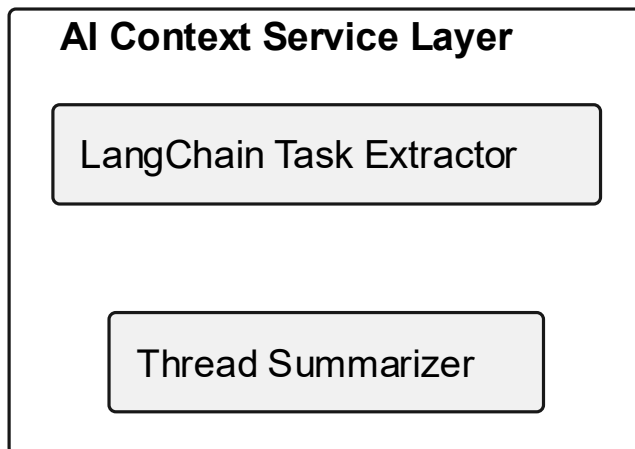
Layer 2: Application Services Layer

- **Subsystems:**
 - **Core API Subsystem:** Spring Boot application handling core project management, task tracking, and user routing.
 - **Security & Auth Subsystem:** Manages JWT token generation, role validation (Admin, PI, Researcher, Resource Manager), and request filtering.
 - **Resource Lock Subsystem:** Algorithms preventing double-booking of shared hardware, workstations, and software assets.
 - **Real-Time Socket Subsystem:** WebSocket server maintaining active connections for real-time updates and collaborative editing



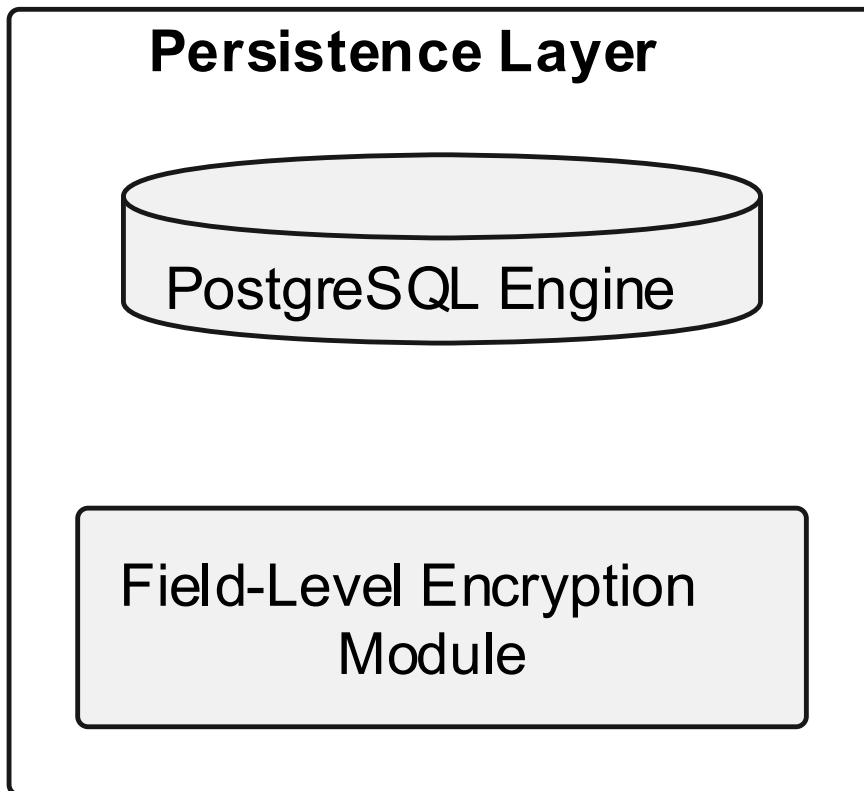
Layer 3: AI Context Service Layer

- **Subsystems:**
 - **Context Processing Subsystem:** LangChain-powered Python microservice responsible for converting raw chat and meeting logs into structured task updates, action items, and deadline suggestions

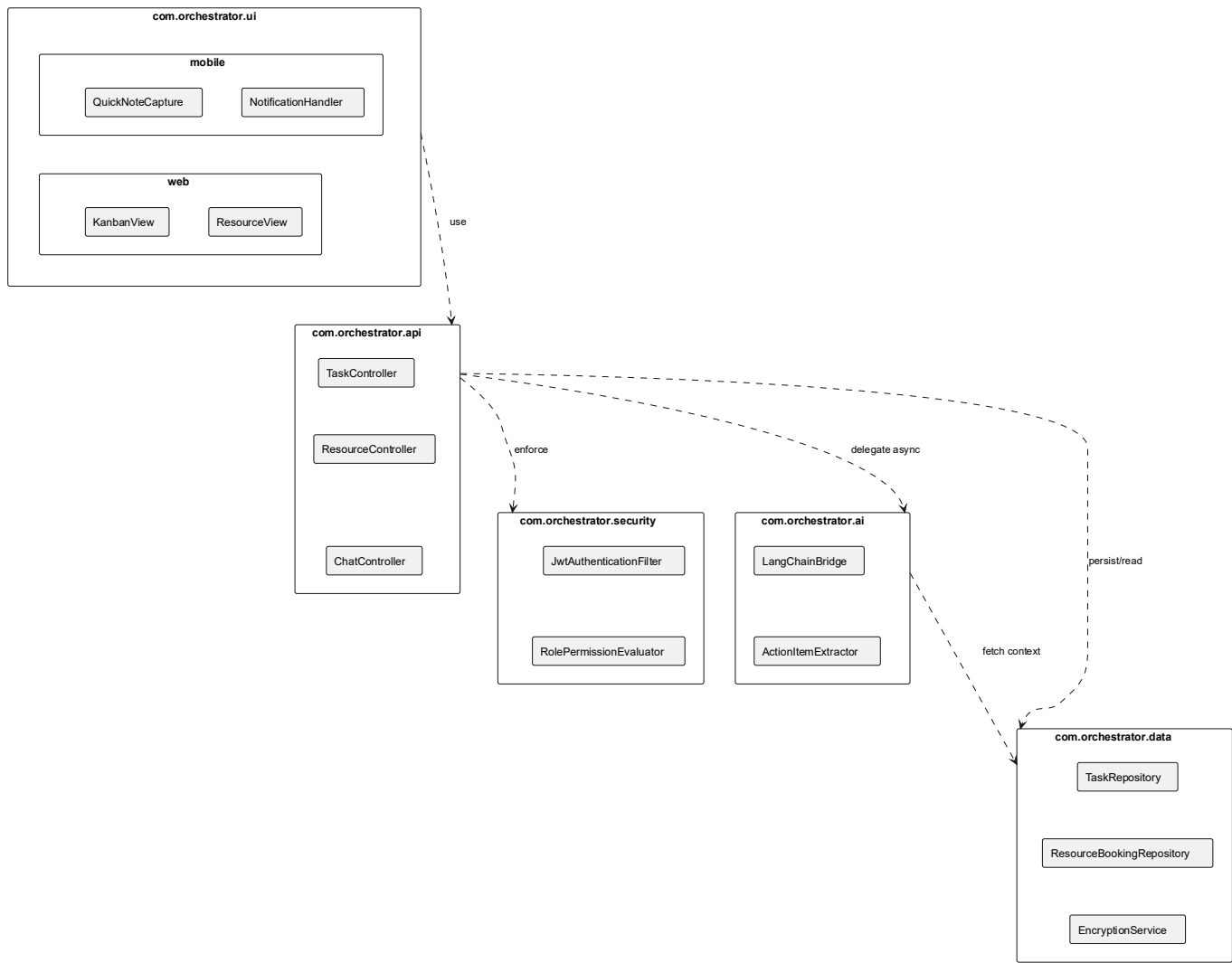


Layer 4: Persistence Layer

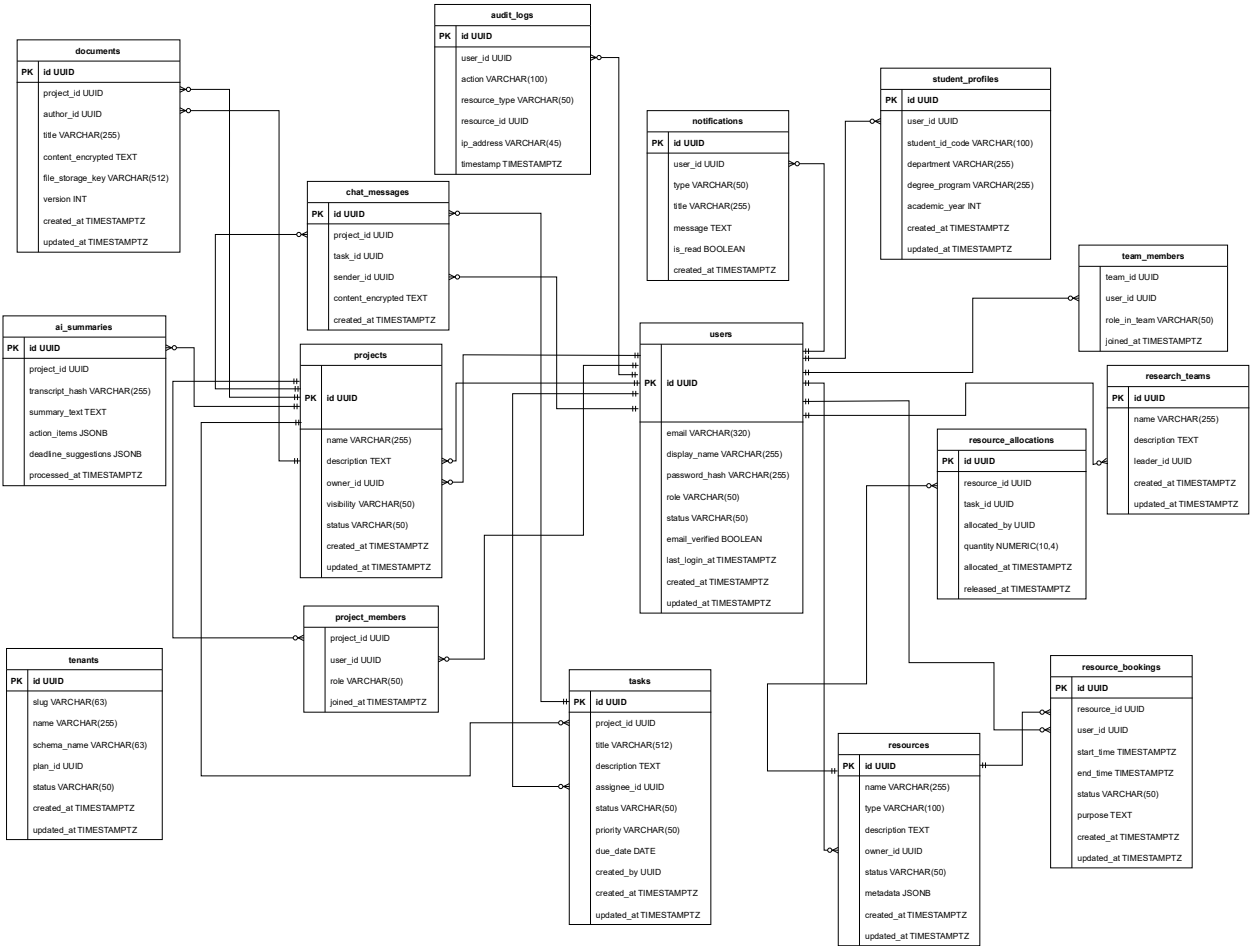
- **Subsystems:**
 - **Database Subsystem:** Single-tenant PostgreSQL instance isolated per research team within Docker containers.
 - **Encryption Subsystem:** Module managing database field-level encryption and key distribution.



Package Diagram

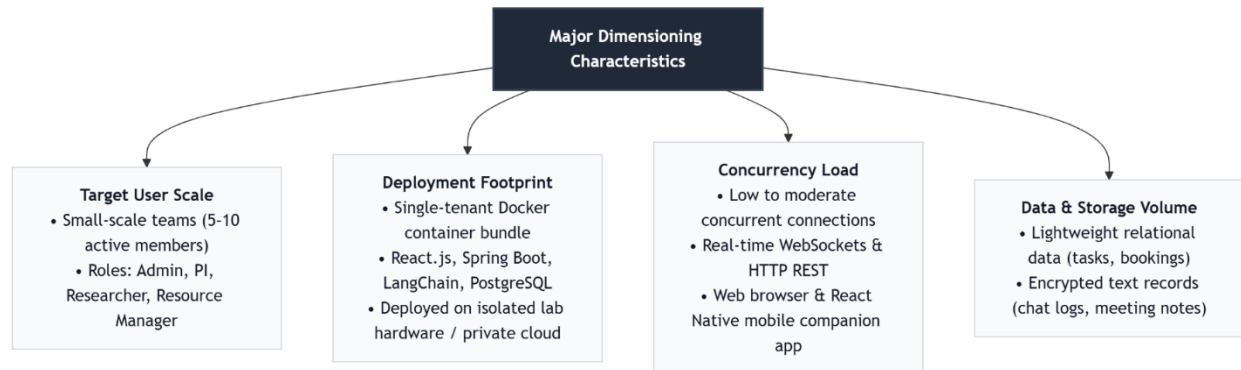


9. Data View



10. Size and Performance

The system is architected as a lightweight, single-tenant Private SaaS platform designed for small, high output academic research labs (typically 5–10 researchers per deployment) running on isolated local infrastructure.



Target Performance Constraints

- **Real-time Synchronization Latency:** The WebSocket server must broadcast interactive Kanban board updates and collaborative rich-text document edits with sub-second latency across active client devices.
- **Resource Locking Transaction Time:** The resource-locking algorithm must execute atomic lock acquisition instantly upon booking requests to guarantee zero double-bookings of shared assets (e.g., lab GPUs, high-performance workstations).
- **Asynchronous AI Engine Processing:** The independent LangChain Context Engine must process chat logs and long discussion threads in the background without blocking core API responsiveness or user workflows.
- **Encryption Overhead Maintenance:** End-to-end encryption at rest and during transit must maintain high throughput with acceptable computational overhead relative to standard unencrypted cloud storage.
- **Notification Delivery:** Mobile push notifications for urgent task updates, mentions, and booking confirmations must dispatch within 1–2 seconds of trigger events.

11. Quality

This section describes how the software architecture contributes to the non-functional capabilities of the system, specifically focusing on usability, portability, reliability, and security.

- **Security and Privacy (Primary Quality Attribute):** The entire architecture is built around preserving data privacy. By utilizing a single-tenant deployment model and AES-256-GCM encryption, the architecture ensures that a laboratory's intellectual property remains completely isolated and secure from third-party data breaches.
- **Usability:** The architecture supports high usability by separating the complex server configuration (handled by Institute Admins) from the daily research interface. The provision of a React Native mobile companion app ensures that researchers can access push notifications and capture notes intuitively while active at the laboratory bench, replacing error-prone spreadsheets with a clean, role-specific UI.
- **Portability:** The strict adherence to Docker containerization ensures high portability. The system can be deployed identically across any standard server environment (Linux, Windows, or macOS) that supports the Docker Engine, allowing research labs to easily migrate the system between local hardware and private cloud instances without code modifications.
- **Reliability and Fault Tolerance:** The polyglot microservices architecture heavily contributes to system reliability through graceful degradation. Because the services are decoupled, an unexpected failure in the Python-based AI Context Engine will not crash the Node.js Socket Server; researchers can continue to chat and collaborate in real-time while the AI service automatically restarts.

12. References

- [1] Reuben LIMS, "Why Spreadsheets Struggle as Laboratories Grow," *Reuben Digital*. [Online]. Available: reubendigital.co.uk
- [2] React Native, "A framework for building native apps using React," *React Native*. [Online]. Available: reactnative.dev
- [3] LangChain AI, "LangChain Documentation," *LangChain*. [Online]. Available: python.langchain.com
- [4] Atlassian, "What is Trello?". [Online]. Available: trello.com
- [5] Slack Technologies, "What is Slack?". [Online]. Available: slack.com

- [6] C. Richardson, *Microservices Patterns: With examples in Java*. Shelter Island, NY, USA: Manning Publications, 2018.
- [7] D. Merkel, "Docker: lightweight Linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, no. 239, p. 2, Mar. 2014.
- [8] National Institute of Standards and Technology (NIST), "Advanced Encryption Standard (AES)," Federal Information Processing Standards Publication (FIPS) 197, Nov. 2001. [Online]. Available: <https://csrc.nist.gov/publications/detail/fips/197/final>
- [9] I. Fette and A. Melnikov, "The WebSocket Protocol," Internet Engineering Task Force (IETF), Request for Comments: 6455, Dec. 2011. [Online]. Available: <https://rfc-editor.org/rfc/rfc6455.html>
- [10] C. A. Ellis and S. J. Gibbs, "Concurrency control in groupware systems," *SIGMOD Rec.*, vol. 18, no. 2, pp. 399–407, Jun. 1989.
- [11] M. Stonebraker and L. A. Rowe, "The design of POSTGRES," in *Proc. 1986 ACM SIGMOD Int. Conf. Management of Data*, Washington, D.C., USA, 1986, pp. 340–355.